



*Faculty of Business
Alexandria University
Management Information Systems Department*



Manzili

A Multi-Vendor E-Commerce Platform for Handmade Items

Submitted By:

Yossef Mohamed El-Sayed

Nour Mohamed Esmat

Yousef Mohamed Abdel-Hakem

Mahmoud Ahmed Mahmoud

Marawan Mahmoud Abdelsalam

Mohamed Abdel-Kader Abd-Rabo

Abdelrhman Mohammed Abdelhamid

Ahmed Elsayed Ahmed

Adham Salah Elsayed

Mahmoud Nagi Nagi

Ali Elsayed Ahmed

Yahia Mohamed Shehata

Fares Haitham Ahmed

Supervised By: Dr. Abeer Amer

Academic Year: 2025 / 2026

Semester: Spring

Acknowledgement

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr. Abeer Amer, for their invaluable guidance, continuous support, and encouragement throughout the development of this project. Their expertise and constructive feedback have been instrumental in shaping this work.

I would also like to thank the faculty and staff of Management Information Systems at Alexandria University for providing the resources and academic environment necessary to complete this graduation project.

Special thanks go to my family and friends for their unwavering moral support, patience, and encouragement during the challenging phases of this project.

Finally, I am grateful to all those who directly or indirectly contributed to making this project a success. Your support has been deeply appreciated.

Special thanks to:

prof. Dr. Ghada El Khayat

Dr. Yasser Abdel Ghaffar

Dr. Abeer Amer

Dr. Esraa Aboulsafa

Assistant lecturer. Mahmoud El Manakhly

Abstract

This report presents the design and development of Manzili, a specialized multi-vendor e-commerce platform dedicated exclusively to handmade products. The platform serves as a bridge connecting skilled Egyptian artisans and makers with customers who seek and value the uniqueness of handcrafted goods, addressing the lack of a dedicated marketplace tailored to the specific needs of the handmade industry.

The system is built using a modern full-stack technology approach, with Next.js and Tailwind CSS on the frontend, ASP .NET Core serving the backend RESTful API, and PostgreSQL managed through Supabase as the database layer. The application is deployed using Docker and AWS, and is accessible via both a web browser and a native Android mobile application powered by WebView. The platform further integrates several payment gateways for secure payment processing, Bosta for delivery logistics, and the Gemini API, DeepSeek, JinaAI for AI-powered features.

Manzili supports three user roles — customers, vendors, and administrators — and offers a dual-model storefront approach: vendors can list ready-to-buy products in a standard gallery, as well as receive and manage custom order requests from customers. Additional features include direct buyer-seller chat, order tracking, and a vendor dashboard for store and product management.

The development process followed a structured software engineering approach covering system analysis, requirements gathering, database and API design, frontend and backend implementation, and comprehensive testing. The result is a scalable, secure, and user-friendly platform that uniquely positions itself within the Egyptian market by combining handmade focus, vendor autonomy, mobile accessibility, and custom order support in a single ecosystem.

Table of Contents

Acknowledgement	3
Abstract	4
Table of Contents	5
List of Figures	9
Chapter 1: Introduction	10
1.1 Project Background	10
1.2 Problem Statement	10
1.3 Project Objectives	11
1.4 Project Scope	11
The platform supports the following user roles:	12
Core modules:	12
1.5 Report Structure	12
Chapter 2: Literature Review & Related Work	13
2.1 Overview of Related Systems	13
Fair Trade Egypt	13
Kanbkam Egypt	13
Imakethis	14
2.2 Review of Technologies Used	14
Frontend Development	14
Backend Development	14
Database	15
Infrastructure & Deployment	15
Security & Networking	15
Mobile Application	16
Design & Planning Tools	16
Third-Party Services & Integrations	16
2.3 Comparison Table	18
Opportunity for Innovation	18
2.4 Summary	19
Chapter 3: System Analysis & Requirements	20
3.1 Business Model Canvas	20
3.2 Data Flow Diagram	21
Data Flow Diagram — Context Diagram	21
External Entities:	21
3.3 Data Flow Diagram — Level 0	22
DFD Level 0 — External Entities & Process Interaction	23
3.4 Use Cases Diagram	26
System Boundary	26
Use Cases (Ovals inside the system)	27
Actor-to-Use-Case Associations	27
Include and Extend Relationships	28

Included use cases in this diagram-----	28
Extended use case in this diagram-----	29
Business Process Implied by the Use Case Diagram -----	29
3.5 Activity Diagram-----	30
Activity Diagram Description: Manzili Order Flow-----	31
Start Node-----	31
Shared Section -----	32
Ending Node: -----	33
Decision Points:-----	33
Merge Points:-----	33
Chapter 4: System Design -----	34
4.1 System Architecture -----	34
Architecture layers:-----	35
4.2 Database Design (ERD)-----	36
4.2.1 User & Identity Entity Group-----	37
4.2.1.1 Entities -----	37
4.2.1.2 Relationships -----	38
4.2.2 Product & Category Entity Group -----	38
4.2.2.1 Entities -----	38
4.2.2.2 Relationships -----	39
4.2.3 Order & Fulfillment Entity Group-----	40
4.2.3.1 Entities -----	40
4.2.3.2 Relationships -----	41
4.2.4 Custom Request Entity Group-----	42
4.2.4.1 Entities -----	42
4.2.4.2 Relationships -----	42
4.2.5 Communication Entity Group-----	43
4.2.5.1 Entities -----	44
4.2.5.2 Relationships -----	44
4.2.6 Financial Operations Entity Group -----	45
4.2.6.1 Entities -----	45
4.2.6.2 Relationships -----	46
4.3 Logical Schema Design -----	47
4.3.1 User & Identity Tables -----	48
4.3.1.1 Person Table -----	48
4.3.1.2 Person_Permissions Table -----	48
4.3.1.3 EndUser Table -----	48
4.3.1.4 Seller Table -----	49
4.3.1.5 System_Admin Table -----	49
4.3.1.6 Addresses Table-----	49
4.3.2.7 Addresses_and_Contact_PhoneNumber Table -----	50
4.3.1.8 Notification Table -----	50
4.3.1.9 Follow Table -----	50
4.3.2 Product & Category Tables -----	50
4.3.2.1 Category Table -----	50
4.3.2.2 Product Table-----	51
4.3.2.3 Product_Images Table -----	51
4.3.2.4 Wishlist Table-----	51
4.3.2.5 Wishlist_Contain Table -----	51
4.3.2.6 Cart Table-----	52
4.3.2.7 Cart_Contain Table-----	52
4.3.2.8 Reviewing And Rating Table -----	52
4.3.3 Order & Fulfillment Tables -----	53
4.3.3.1 Order Table -----	53

4.3.3.2 Standrad_Order Table	53
4.3.3.3 Customized_Order	53
4.3.3.4 Consist of (Order Item) Table	54
4.3.3.5 Shipment Table	54
4.3.3.6 Returns Table	54
4.3.3.7 Proof_As_Images Table	55
4.3.4 Custom Request Tables	55
4.3.4.1 Custom_Request Table	55
4.3.4.2 Visual_Inspiration Table	56
4.3.4.3 Voicememo Table	56
4.3.4.4 Color_Palette Table	56
4.3.5 Communication Tables	57
4.3.5.1 Conversation Table	57
4.3.5.2 Messages Table	57
4.3.6 Financial Operations Tables	58
4.3.6.1 Payout Table	58
4.3.6.2 Coupons Table	58
4.3.6.3 User_Coupons Table	58
4.3.7 Business Process Flow (High-Level)	59
Registration & Identity	59
Product Discovery	59
Standard Order Placement	59
Custom Commissioning	59
Communication	59
Order Fulfillment & Returns	59
Financial Operations	60
4.3.8 What This Schema Represents	60
4.4 API Design	61
4.5 UI/UX Wireframes	61
Chapter 5: Implementation	63
5.1 Development Environment	63
5.2 Frontend Implementation	63
5.3 Backend Implementation	64
5.4 Database Implementation	65
5.5 Active Database vs. the ERD Design	65
5.6 AI Features Implementation	66
5.7 System Walkthrough	67
5.7.1 Global Layout	67
Header (Navbar)	67
Footer	68
5.7.2 Authentication	69
Register (/register)	69
Login (/login)	69
Admin Login (/admin/login)	70
5.7.3 Buyer Interface	71
Home (/)	71
Shop (/shop)	72
Storefront (/shop/[storeSlug])	72
Product Details (/shop/[storeSlug]/[productSlug])	73
Cart & Checkout (/cart and /checkout)	74
Wishlist (/wishlist)	74

My Orders (/orders) -----	75
Returns (/returns) -----	75
Profile (/profile) -----	76
Custom Requests Hub (/custom-products) -----	76
New Custom Request (/custom-products/new) -----	77
Request View (/custom-products/[requestId]) -----	77
Offer Negotiation (/custom-products/[requestId]/offers/[offerId]) -----	78
Collaboration Workspace (/custom-products/[requestId]/offers/[offerId]/workspace) -----	79
Become a Vendor (/become-a-vendor) -----	80
Informational Pages -----	81
5.7.4 Vendor Dashboard -----	82
Dashboard (/store) -----	82
Add Product (/store/products/new) -----	83
Manage Products (/store/products) -----	83
Store Orders (/store/orders) -----	84
Custom Orders (/store/custom-orders) -----	85
Coupons (/store/coupons) -----	85
Wallet (/store/wallet) -----	86
Returns (/store/returns) -----	87
Warehouses (/store/warehouses) -----	87
Settings (/store/settings) -----	88
5.7.5 Admin Panel -----	88
Admin Dashboard (/admin) -----	89
Chapter 6: Testing & Evaluation -----	90
6.1 Testing Strategy -----	90
6.2 Cookie-Based Prototyping on Localhost -----	90
6.3 Integration Testing -----	90
6.4 Backend Migration, API Testing, and Supabase -----	90
6.5 Results -----	91
Chapter 7: Conclusion & Future Work -----	92
7.1 Summary of Achievements -----	92
7.2 Limitations -----	93
7.3 Future Enhancements -----	94
7.4 Closing Remarks -----	94
Appendices -----	96
Appendix A: -----	96

List of Figures

<i>Figure 1: Business Model Canvas</i>	20
<i>Figure 2: DFD (Context Diagram)</i>	21
<i>Figure 3: Diagram (Level 1)</i>	22
<i>Figure 4: Use Cases Diagram</i>	26
<i>Figure 5 Activity Diagram</i>	30
<i>Figure 6: System Architecture Overview</i>	34
<i>Figure 7: Entity Relationship Diagram</i>	36
<i>Figure 8: Logical Schema</i>	47

Chapter 1: Introduction

1.1 Project Background

Handmade products represent a significant and growing segment of the retail market, valued by customers for their uniqueness, craftsmanship, and personal touch. In Egypt, the handmade industry is rich with talented artisans and small-scale makers who produce high-quality, unique goods ranging from accessories and home décor to clothing and art pieces. However, despite this wealth of talent, the industry remains highly fragmented and largely informal.

Most artisans currently rely on social media platforms or general e-commerce websites to reach customers. While social media offers visibility, it requires significant ongoing marketing effort to find the right audience and rarely provides the infrastructure needed for secure transactions, order management, or business growth. On the other hand, general e-commerce platforms are designed primarily for mass-produced goods, where products are often ranked and promoted based on price rather than craftsmanship or uniqueness. As a result, handmade products are forced to compete directly with cheaper, factory-made alternatives, which undermines the perceived value of artisanal work.

This project, Manzili, was conceived to address this gap by creating a specialized multi-vendor e-commerce platform — available as both a website and a mobile application — dedicated strictly to handmade products. Manzili aims to serve as a bridge connecting skilled artisans and makers with customers who specifically seek and value handcrafted goods, providing both parties with a centralized, purpose-built marketplace.

1.2 Problem Statement

The Egyptian handmade industry suffers from what can be described as a "visibility paradox": artisans produce high-value, unique items, yet they are often forced to market and sell these items on platforms that were never designed for handmade goods.

This creates several core problems:

Lack of a specialized marketplace: General e-commerce platforms do not provide categories, search filters, or browsing experiences tailored to handmade and artisanal products, making it difficult for customers to discover them.

Unfair competition with mass-produced goods: On general marketplaces, pricing algorithms and rankings tend to favor cheaper, mass-produced items, overshadowing the value of craftsmanship and overshadowing handmade sellers.

High marketing burden on makers: Relying on social media to reach customers requires artisans to dedicate significant time and resources to marketing, which many small-scale makers cannot sustain alongside production.

Lack of personalization and trust: Without a dedicated ecosystem that understands the unique nature of handmade work — including customization requests, production timelines, and the story behind each product — makers struggle to build lasting relationships with customers who appreciate premium, handcrafted goods.

Collectively, these issues lead to financial instability for artisans and limit the growth potential of local, home-grown handmade brands. There is a clear need for a centralized, specialized platform that connects handmade product makers with the customers who value their work.

1.3 Project Objectives

The main objectives of this project are:

- To design and develop a multi-vendor e-commerce web application dedicated exclusively to handmade products.
- To provide artisans and makers with a dedicated platform and storefront to showcase, manage, and sell their handmade goods without competing against mass-produced items.
- To implement product categories and search/filtering features tailored specifically to handmade and artisanal goods, improving product discoverability for customers.
- To develop a secure and scalable backend system supporting multiple vendors, product listings, order management, and secure payment processing.
- To design a responsive and intuitive user interface that allows customers to easily browse, search, and purchase handmade products.
- To provide vendors with tools to manage their shops, including adding/editing products, tracking orders, and viewing sales analytics.
- To reduce the marketing burden on individual artisans by providing a centralized platform where the target audience actively seeks handmade products.

1.4 Project Scope

This project focuses on the design and development of Manzili as a multi-vendor e-commerce platform consisting of a frontend (customer-facing web application) and a backend (server, database, and vendor management system).

The platform supports the following user roles:

- Customers — who can browse products by category, search and filter handmade items, view product details, add items to a cart, place orders, and track their purchases.
- Vendors (Makers/Artisans) — who can register, set up their own storefronts, list and manage their handmade products, and manage incoming orders.
- Administrators — who oversee the platform, manage vendor approvals, monitor categories, and ensure overall platform quality and compliance.

Core modules:

- User authentication and account management (for customers and vendors).
- Vendor storefront and product management module.
- Product browsing, search, and category-based filtering tailored to handmade goods.
- Shopping cart and order placement system.
- Order management and tracking for both customers and vendors.
- Admin dashboard for platform oversight.

1.5 Report Structure

This report is organized as follows: Chapter 2 presents a review of related platforms and the technologies used in building Manzili, along with a comparison highlighting how it differs from general e-commerce solutions.

Chapter 3 covers the system analysis, including functional and non-functional requirements, use case diagrams, and user stories for customers, vendors, and administrators.

Chapter 4 details the system design, including the overall system architecture, database design (ERD), API design, and UI/UX wireframes for both the customer and vendor experiences.

Chapter 5 describes the implementation of the frontend and backend components, along with the database setup.

Chapter 6 presents the testing strategy and results, covering unit, integration, and user acceptance testing.

Finally, Chapter 7 concludes the report with a summary of achievements, project limitations, and recommendations for future enhancements.

Chapter 2: Literature Review & Related Work

2.1 Overview of Related Systems

Several platforms currently operate within the Egyptian market with the goal of connecting creators and consumers. While each of these platforms shares some similarities with Manzili, they each fall short in different ways, leaving a gap that Manzili aims to fill.

Fair Trade Egypt

Fair Trade Egypt is a long-standing social enterprise that focuses on traditional heritage crafts and ethical sourcing. It provides a vital lifeline for marginalized artisans by giving their products access to a wider market.

However, Fair Trade Egypt operates as a retailer rather than an open marketplace —

artisans sell their products to the organization, which then resells them. As a result, artisans lack the autonomy to manage their own digital storefronts, build individual brand identities, or interact directly with customers.

Additionally, the product catalog is strictly limited to traditional heritage crafts, excluding modern or contemporary handmade products.



Kanbkam Egypt

Kanbkam is a price-comparison and aggregation engine that tracks prices across Egypt's major e-commerce platforms, including Amazon, Noon, and Jumia. When a user searches for a handmade item, Kanbkam pulls listings from these mass-market platforms and ranks them primarily by price.

This approach treats handmade items as commodities, forcing unique, time-intensive crafts to compete directly with cheap, factory-made alternatives. This price-driven model devalues the time, skill, and uniqueness behind artisanal work, and offers no dedicated space or recognition for handmade products.



Imakethis

imakethis is a specialized digital marketplace built specifically for Egyptian crafters. Similar to Manzili, it focuses on showcasing local talent and provides artisans with independent storefronts and categories dedicated to handmade goods. However, imakethis primarily operates as a standard retail gallery, where products are listed and sold as-is. It does not provide a dedicated infrastructure for customers who wish to collaborate with makers on custom or modified versions of a product, nor does it offer a mobile application or direct communication channel between buyers and sellers.



2.2 Review of Technologies Used

This section reviews the core technologies, frameworks, and services used in the development of Manzili, along with a brief justification for each choice in the context of building a scalable, multi-vendor e-commerce platform.

Frontend Development

- **Next.js:** A React-based framework used for building the customer-facing web application and vendor dashboards. Next.js was chosen for its support of server-side rendering (SSR) and static site generation (SSG), which improve page load speed and search engine optimization (SEO) — both critical for an e-commerce platform where product discoverability directly impacts sales.
- **Tailwind CSS:** A utility-first CSS framework used for styling the user interface. Tailwind enables rapid development of a consistent, responsive, and visually appealing design system without writing extensive custom CSS, which helped maintain design consistency across the customer storefront, vendor dashboard, and admin panel.



Backend Development

- **.NET (ASP.NET Core):** A cross-platform backend framework developed by Microsoft, used to build the server and RESTful API that powers Manzili. ASP.NET Core was selected for its high performance, strong typing through C#, and robust built-in support for building scalable web APIs. Its structured architecture and extensive ecosystem make it well-suited for handling the platform's core business logic, including authentication, order processing, and third-party service integrations.



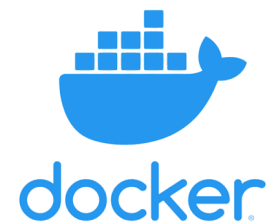
Database

- PostgreSQL: A relational database management system used to store structured data such as users, products, orders, vendors, and categories. PostgreSQL was chosen for its reliability, strong support for relational data (important given the relationships between vendors, products, customers, and orders), and its compatibility with Supabase.
- Supabase: An open-source backend-as-a-service platform built on top of PostgreSQL, used to manage the database, authentication, and real-time features (such as live chat or order status updates). Supabase accelerates development by providing ready-made APIs, authentication, and storage on top of a PostgreSQL database.



Infrastructure & Deployment

- Docker: Used to containerize the application components (frontend, backend, and supporting services), ensuring consistency across development, testing, and production environments, and simplifying deployment.
- AWS (Amazon Web Services): Used to host and deploy the containerized application, providing scalable cloud infrastructure, reliable uptime, and the ability to scale resources based on platform traffic and demand.
- Vercel: Used to deploy and host the Next.js frontend application. Vercel was selected for its seamless integration with Next.js, offering automatic deployments, edge network delivery, and optimized performance for server-side rendered and statically generated pages.



Security & Networking

Cloudflare (Proxy Tunnel & HTTPS Encryption):

Cloudflare is used as a reverse proxy and security layer between the frontend and backend.

Rather than exposing the backend server directly to the public internet, all traffic is routed through a Cloudflare proxy tunnel, which handles HTTPS encryption, hides the origin server's IP address, and provides an additional layer of protection against direct attacks on the backend infrastructure. This ensures all data transmitted between the client and the server is encrypted in transit without requiring manual SSL certificate management on the application servers.



Mobile Application

- Android (WebView-based): The mobile application is built as an Android app powered by a WebView component, which wraps the existing web application within a native Android shell. This approach allows Manzili to offer a mobile app experience while reusing the existing Next.js frontend, reducing development time and ensuring consistency between the web and mobile experiences.



Design & Planning Tools

- draw.io: Used during the design phase to create system diagrams including use case diagrams, data flow, system architecture diagrams, and activity diagrams, supporting clear visual communication of the system design throughout development.
- ERDPlus: Used specifically during the database design phase to design and produce the Entity Relationship Diagram (ERD) ERDPlus provided a structured environment for modeling entities, attributes, relationships, and cardinalities in standard ERD notation, serving as the primary tool for conceptual database design.
- dbdiagram.io: Used during the database design phase to design, visualize, and document the relational schema of the Manzili database. It provided a clear visual representation of all tables, columns, data types, and foreign key relationships, serving as the primary tool for producing the logical schema diagrams included in this documentation.
- Apidog: Used alongside Postman as an all-in-one API development tool for designing, documenting, and testing API endpoints, helping maintain clear and up-to-date API documentation for the backend services throughout development.



draw.io



Third-Party Services & Integrations

In addition to the core stack, Manzili integrates with several third-party services to provide essential e-commerce functionality:

- Stripe: A payment processing service integrated to handle secure online payments, allowing customers to pay for orders using credit/debit cards and other supported payment methods.



- **Kashier:** An Egyptian payment gateway integrated to complement Stripe by supporting local payment methods commonly used by Egyptian consumers. Kashier enables customers to complete transactions using locally preferred options that are not covered by international card processors, broadening the platform's payment accessibility within the Egyptian market.
- **Bosta:** A delivery and logistics service API integrated to manage order shipping, generate shipping labels, and track deliveries for orders placed through the platform.
- **Gemini API (LLM/AI Services):** A large language model API used to power AI-driven features within the platform.
- **DeepSeek V4:** A large language model used alongside the Gemini API to power AI-assisted features within the Manzili platform.
- **Jina AI:** An AI-powered embedding service used to generate semantic vector representations of products and search queries for the platform's AI semantic search feature. Jina AI converts text into high-dimensional vector embeddings that are stored in the database using pgvector, enabling meaning-based similarity search across the product catalog rather than relying on exact keyword matching.



bosta



Gemini



2.3 Comparison Table

To clearly highlight the differences between the existing platforms and the proposed Manzili system, the table below compares each platform across several key features relevant to handmade product marketplaces.

Feature	Fair Trade Egypt	Kanbkam	I Make This	Manzili
Product Catalog	✓	✓	✓	✓
Handmade Focus	✓	✗	✓	✓
Seller Stores	✗	✓	✓	✓
Mobile App	✗	✓	✗	✓
Custom Orders	✗	✗	✗	✓
Direct Chat	✗	✗	✗	✓

Opportunity for Innovation

As shown in the comparison above, none of the existing platforms combine a handmade-focused catalog, independent seller stores, a mobile application, support for custom orders, direct buyer-seller communication, and integrated payment and delivery services within a single ecosystem. This represents a clear opportunity for Manzili to differentiate itself in the Egyptian market.

Manzili addresses this gap through a dual-model storefront approach: each vendor's store offers both a gallery of ready-to-buy items for immediate purchase, and a custom request portal where customers can collaborate directly with makers on bespoke or personalized products. This dual model not only enhances scalability for vendors — allowing them to serve both walk-in buyers and dedicated custom clients — but also uniquely positions Manzili as a marketplace built specifically around the needs of artisans and the customers who value their craft.

2.4 Summary

This chapter provided a comprehensive overview of the existing landscape of handmade product marketplaces in the Egyptian market, the technologies selected for building Manzili, and a direct feature comparison between competing platforms.

The review of related systems in Section 2.1 revealed that while platforms such as Fair Trade Egypt, Kanbkam, and imakethis each address parts of the problem, none of them provide a complete, dedicated ecosystem for handmade product commerce. Fair Trade Egypt lacks vendor autonomy, Kanbkam treats handmade goods as price-driven commodities, and imakethis, while the closest in concept, lacks custom order support and direct buyer-seller communication.

Section 2.2 outlined the technology stack chosen for Manzili, which combines modern, industry-proven tools — including Next.js, Tailwind CSS, ASP.NET Core, PostgreSQL, Supabase, Docker, Vercel, and AWS — alongside key third-party integrations for payments (Stripe and Kashier), delivery logistics (Bosta), media hosting (Cloudinary), AI-powered features (Gemini API, DeepSeek V4, and Jina AI for semantic search), and network security (Cloudflare). This stack was selected to ensure scalability, development efficiency, and a high-quality user experience across both web and mobile platforms.

The comparison table in Section 2.3 confirmed that Manzili is the only platform among those reviewed to offer the full combination of a handmade-focused catalog, independent vendor storefronts, a mobile application, custom order support, and direct chat between buyers and sellers.

Chapter 3: System Analysis & Requirements

3.1 Business Model Canvas

The Business Model Canvas (BMC) provides a high-level strategic view of Manzili's business logic, showing how value is created, delivered, and captured across the platform.

It organizes the core business components--- including key partners, key activities, value propositions, customer segments, channels, customer relationships, cost structure, and revenue streams --- into a single visual framework that supports alignment between technical planning and business objectives.

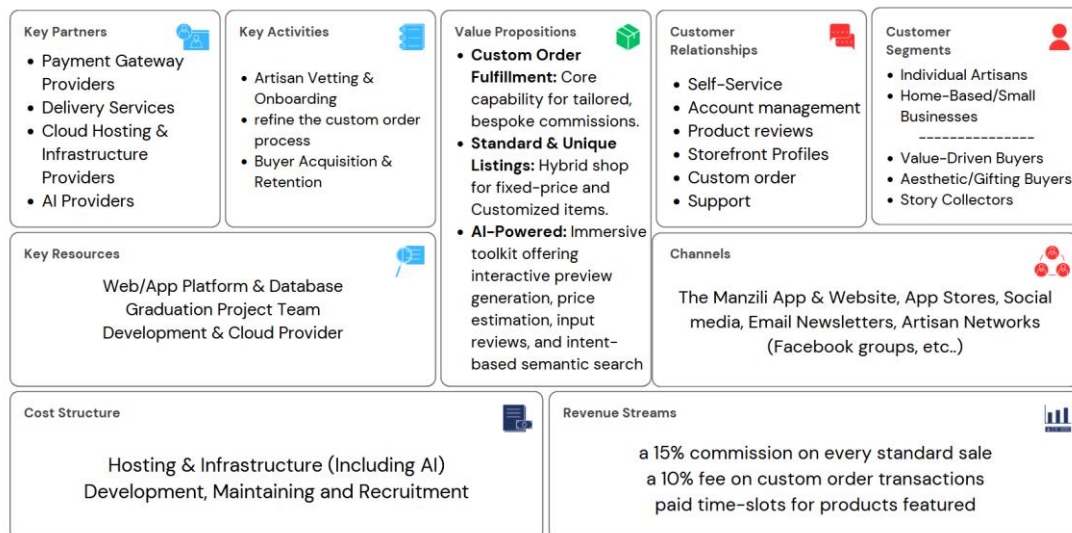


Figure 1: Business Model Canvas

The complete and in-depth Business Model Canvas is accessible from [here](#) or by scanning QR code.



3.2 Data Flow Diagram

Data Flow Diagram — Context Diagram

The Context Diagram provides the highest-level view of the Manzili system. It treats the entire platform as a single process and shows only the external entities that interact with it and the data flowing between them.

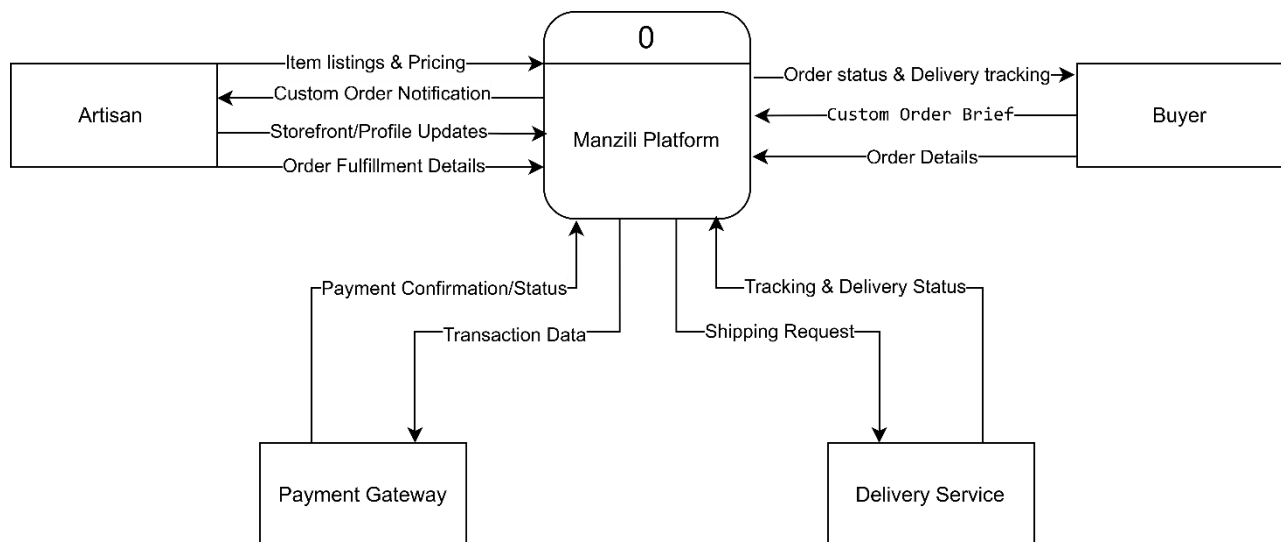


Figure 2: DFD (Context Diagram)

External Entities:

Buyer — sends login credentials, browse/search queries, orders, payments, custom requests, chat messages, and reviews. Receives product listings, order confirmations, order status updates, and chat responses.

Artisan — sends login credentials, product listings, order status updates, custom order responses, and chat messages. Receives order notifications, custom requests, customer reviews, and chat messages.

Payment Gateway — receives payment details. Returns payment confirmation or failure.

Delivery Service — receives shipment requests. Returns delivery tracking information.

3.3 Data Flow Diagram — Level 0

The Level 0 DFD expands the single system process from the context diagram into the major functional sub-processes within Manzili, showing how data flows between them and the data stores involved.

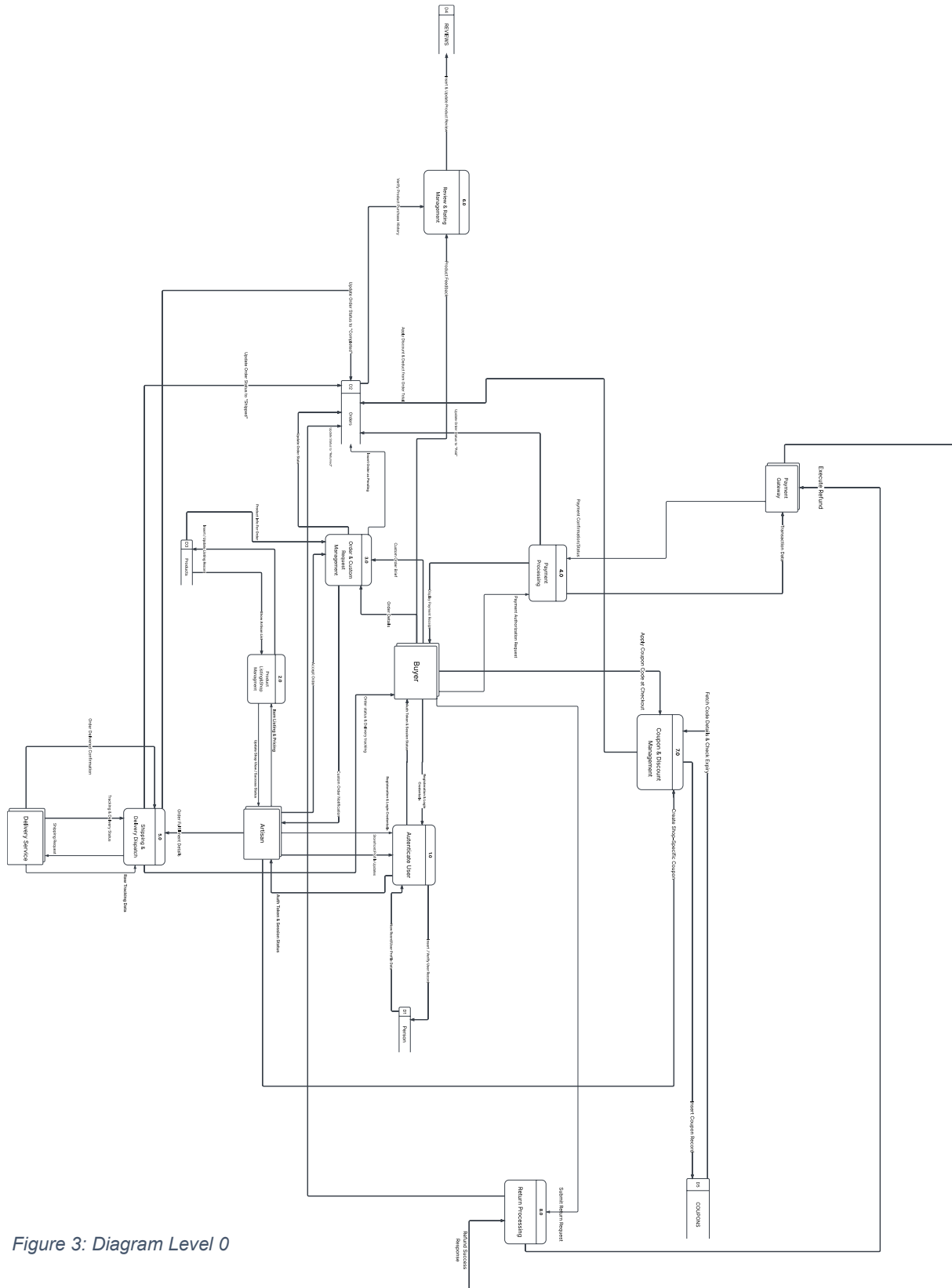


Figure 3: Diagram Level 0

DFD Level 0 — External Entities & Process Interaction

The following describes the data flows between each external entity and the system processes identified in the Context DFD, including the corresponding database operations triggered by each interaction.

1. Buyer

➤ Data Sent to the System:

- Process 1.0 — Authenticate User: Registration & Login Credentials.
 - Database Action: The system verifies or inserts the account credentials into **D1: Person**.
- Process 3.0 — Order & Custom Request Management: Order Details and Custom Order Brief.
 - Database Action: The system executes an Insert Order as Pending operation into **D2: Orders**.
- Process 4.0 — Payment Processing: Payment Authorization Request.
- Process 6.0 — Review & Rating Management: Product Feedback.
 - Database Action: The system queries D2: Orders to verify purchase history, then persists the submission via Insert & Update Product Review into **D4: Reviews**.
- Process 7.0 — Coupon & Discount Management: Apply Coupon Code at Checkout.
 - Database Action: The system queries D5: Coupons to fetch code details and check expiry, then applies the discount and deducts it from the order total in **D2: Orders**.
- Process 8.0 — Return Processing: Submit Return Request.

➤ Data Received from the System:

- Process 1.0 — Authenticate User: Auth Token & Session Status.
- Process 4.0 — Payment Processing: Payment Receipt.
- Process 5.0 — Shipping & Delivery Dispatch: Order Status & Delivery Tracking.

2. Artisan

➤ Data Sent to the System:

- Process 1.0 — Authenticate User: Registration & Login Credentials.
 - Database Action: The system writes or verifies accounts via Insert / Verify User Record into **D1: Person**.
- Process 2.0 — Product Listing & Shop Management: Item Listing & Pricing.
 - Database Action: The system populates the public catalog via Insert / Update Listing Record into **D3: Products**.
- Process 3.0 — Order & Custom Request Management: Accept Order.
 - Database Action: The system executes an Update Order State operation inside **D2: Orders**.
- Process 5.0 — Shipping & Delivery Dispatch: Order Fulfillment Details.
- Process 7.0 — Coupon & Discount Management: Create Shop-Specific Coupon.
 - Database Action: The system commits the promotional record via Insert Coupon Record into **D5: Coupons**.

➤ Data Received from the System:

- Process 1.0 — Authenticate User: Auth Token & Session Status.
- Process 2.0 — Product Listing & Shop Management: Updated Shop View & Success Status.
- Process 3.0 — Order & Custom Request Management: Custom Order Notification.

3. System Admin

➤ Data Sent to the System:

- Process 1.0 — Authenticate User: Registration & Login Credentials.
- Process 7.0 — Coupon & Discount Management: Create Platform-Wide Coupon.

- Database Action: The system tracks global marketplace discounts via Insert Coupon Record into **D5: Coupons**.
- Process 8.0 — Return Processing: Approve / Decline Return Order.

➤ Data Received from the System:

- Process 1.0 — Authenticate User: Auth Token & Session Status.
- Process 8.0 — Return Processing: New Return Request Notification.

4. Payment Gateway

➤ Data Sent to the System:

- Process 4.0 — Payment Processing: Payment Confirmation / Status.
 - Database Action: The system executes an Update Order Status to "Paid" operation inside **D2: Orders**.
- Process 8.0 — Return Processing: Refund Success Response.
 - Database Action: The system executes an Update Order Status to "Returned" operation within **D2: Orders**.

➤ Data Received from the System:

- Process 4.0 — Payment Processing: Transaction Data.
- Process 8.0 — Return Processing: Execute Refund instruction.

5. Delivery Service

➤ Data Sent to the System:

- Process 5.0 — Shipping & Delivery Dispatch: Tracking & Delivery Status, Raw Tracking Data, and Order Delivered Confirmation.
 - Database Action (On Transit): The system executes an Update Order Status to "Shipped" operation within **D2: Orders**.
 - Database Action (On Arrival): The system executes an Update Order Status to "Completed" operation inside **D2: Orders**.

➤ Data Received from the System:

- Process 5.0 — Shipping & Delivery Dispatch: Shipping Request to initiate the delivery process.

3.4 Use Cases Diagram

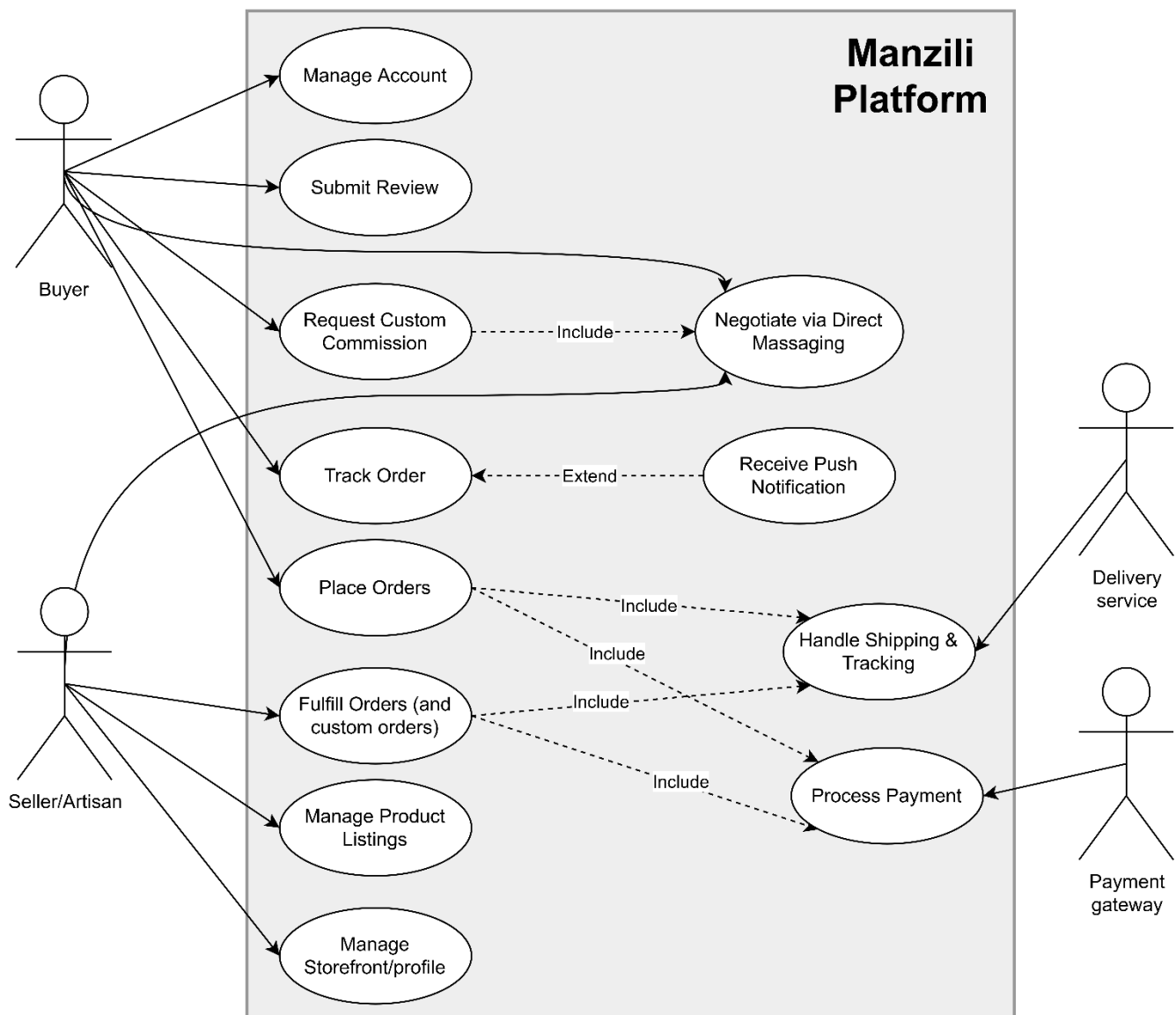


Figure 4: Use Cases Diagram

System Boundary

- System Name: Manzili Platform
- The gray rectangle represents the application boundary.

- All oval shapes inside this boundary are use cases provided by the platform.
- The circles outside the boundary represent external actors/systems interacting with those use cases.

Use Cases (Ovals inside the system)

1. Manage Account: User updates profile/account information and authentication-related settings.
2. Submit Review: Buyer submits rating/review after order completion.
3. Request Custom Commission: Buyer requests a custom-made item or bespoke order.
4. Negotiate via Direct Messaging: Buyer and seller discuss custom request scope, price, and timeline.
5. Track Order: Buyer monitors order status and delivery progress.
6. Receive Push Notification: User receives real-time status/event notifications from the platform.
7. Place Orders: Buyer places standard marketplace orders.
8. Fulfill Orders (and custom orders): Seller processes and completes standard/custom orders.
9. Handle Shipping & Tracking: Shipping lifecycle is handled, and tracking events are updated.
10. Process Payment: Payments are processed for ordering and fulfillment workflows.
11. Manage Product Listings: Seller creates/updates/removes product catalog entries.
12. Manage Storefront/profile: Seller manages store identity (store profile, visual branding, details).

Actor-to-Use-Case Associations

1. Buyer (left, upper)

Associated use cases:

- Manage Account
- Submit Review

- Request Custom Commission
- Track Order
- Place Orders
- Negotiate via Direct Messaging (direct association visible)

2. Artisan (left, lower)

Associated use cases:

- Fulfill Orders (and custom orders)Submit Review
- Manage Product ListingsTrack Order
- Manage Storefront/profile
- Negotiate via Direct Messaging (direct association visible)

3. Delivery service (right side, upper)

Associated integration:

- Handle Shipping & Tracking
- Payment gateway (right side, lower)
- Associated integration:
- Process Payment

Include and Extend Relationships

In UML use case notation:

<<include>> means a mandatory reused sub-function.

<<extend>> means optional/conditional behavior extending a base use case.

Included use cases in this diagram

1. **Request Custom Commission --`<<include>>`--> Negotiate via Direct Messaging**
Meaning: creating a custom commission necessarily involves a negotiation step.
2. **Place Orders --`<<include>>`--> Handle Shipping & Tracking**
Meaning: order placement includes shipping/tracking handling as part of the flow.

3. **Place Orders --`<<include>>`--> Process Payment**
Meaning: payment is a required included function of placing orders.
4. **Fulfill Orders (and custom orders) --`<<include>>`--> Handle Shipping & Tracking**
Meaning: fulfillment includes shipment creation/tracking updates.
5. **Fulfill Orders (and custom orders) --`<<include>>`--> Process Payment**
Meaning: fulfillment also invokes required payment operations (capture/settlement/release).

Extended use case in this diagram

1. **Receive Push Notification --`<<extend>>`--> Track Order**
Meaning: push notifications are conditional/triggered enhancements to tracking behavior (e.g., status changes).

Business Process Implied by the Use Case Diagram

This use case diagram models Manzili as a multi-vendor commerce platform with both standard and custom ordering modes:

1. Buyer journey: Buyers manage accounts, place orders, track deliveries, and submit reviews.
2. Custom commerce capability: Buyers can request custom commissions, with direct negotiation as a required part of the custom workflow.
3. Seller operations: Sellers maintain listings/storefronts and fulfill both standard and custom orders.
4. Core shared services: Ordering and fulfillment both rely on payment processing and shipping/tracking capabilities.
5. External ecosystem integration: The platform connects to payment infrastructure and logistics systems; messaging/notification channels support negotiation and updates.
6. User communication layer: Tracking can be extended through push notifications, improving transparency and delivery visibility.
7. Together, these use cases describe a platform where product discovery, transaction processing, fulfillment execution, and post-purchase feedback are unified, while still supporting a negotiation-heavy custom-order path.

3.5 Activity Diagram

Buyer

Platform

Seller

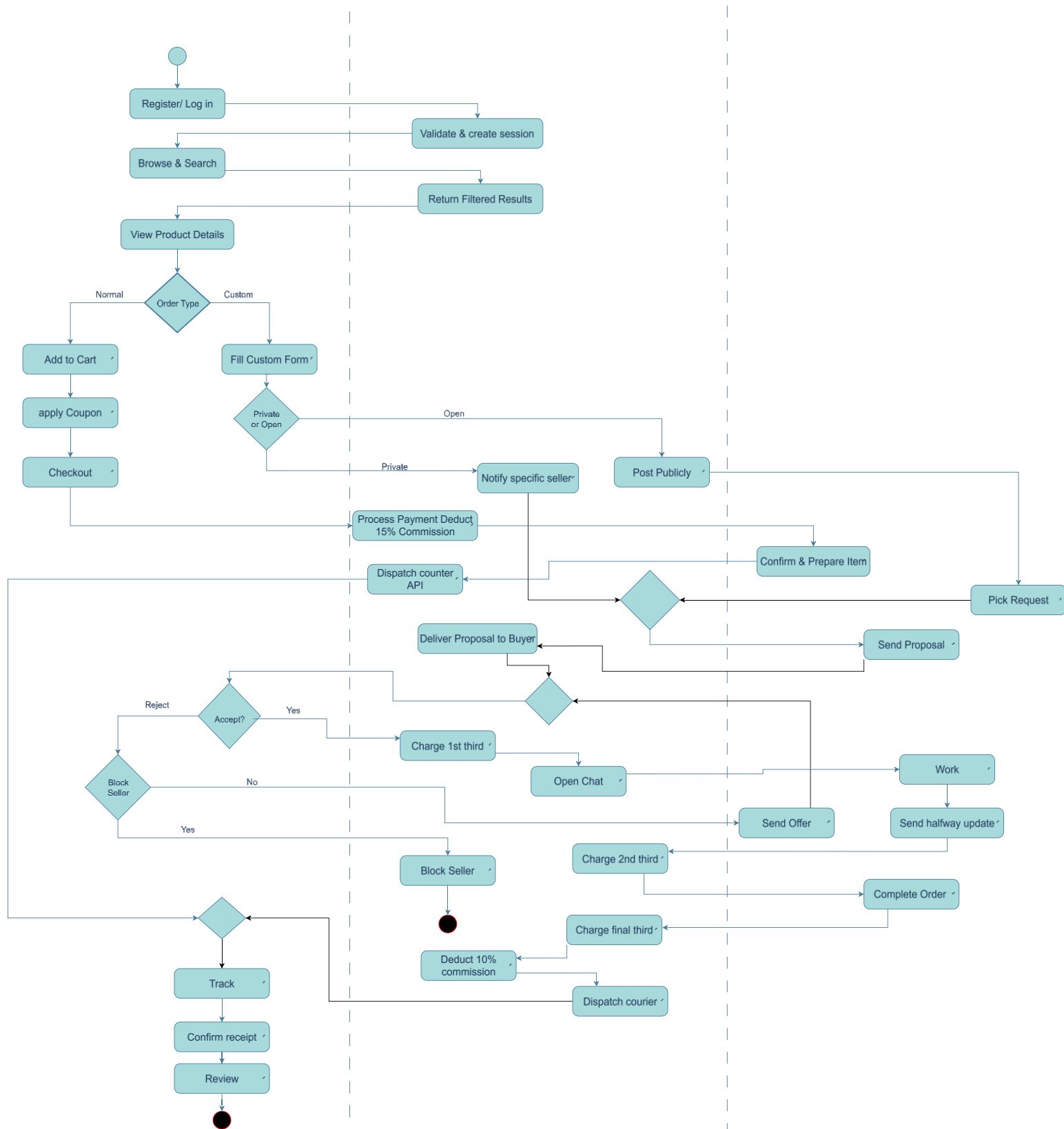


Figure 5 Activity Diagram

Activity Diagram Description: Manzili Order Flow

Start Node

The flow begins at a Start node in the Buyer's lane.

1. The Buyer performs **Register/Log in**
2. Control passes to the Platform, which **performs Validate & create session**.
3. Control returns to the Buyer, who performs **Browse & Search**.
4. The Platform responds with **Return Filtered Results**.
5. The Buyer performs **View Product Details**.
6. This leads to a decision node, **Order Type**, splitting into two branches:
 - a. **Branch 1 - Normal Order**
 - i. The Buyer performs **Add to Cart**.
 - ii. Then **Apply Coupon**.
 - iii. Then **Checkout**.
 - iv. Checkout flows directly into **Process Payment, Deduct 15% Commission** (Platform lane).
 - b. **Branch 2 - Custom Order**
 - i. The Buyer performs Fill **Custom Form**
 - ii. This leads to a decision node, **Private or Open**, with two branches:
 1. **Open**: flow goes to the Platform, which performs **Post Publicly**.
 2. **Private**: flow goes to the Platform, which performs **Notify Specific Seller**.
7. **Pick Request** flows directly into the **merge node** (a diamond positioned between the Platform and Seller lanes) — entering from the right side.
8. From **Notify Specific Seller**, the flow goes straight down and enters the **same merge node** from the left side
9. From this merge node, the flow continues to the Seller, who performs **Send Proposal**.
10. **Send Proposal** flows into the Platform's **Deliver Proposal to Buyer**.

11. **Deliver Proposal to Buyer** flows down into a **second merge node**

12. This second merge node flows up into the Buyer's decision, **Accept?**, with two branches:

- a. **Reject**: flow moves to another decision, **Block Seller?**:
 - i. If **Yes** → flow goes to the Platform, which **performs Block Seller**, ending at a separate **End node**.
 - ii. If **No** → the flow goes right and down into the Seller's **Send Offer**, which then flows back up into the **second merge node** (re-entering the negotiation loop, back to "Deliver Proposal to Buyer" → "Accept?").
- iii. **Accept(Yes)**:
 1. The Platform performs **Charge 1st Third**.
 2. Then **Open Chat** is initiated, flowing into the Seller's lane.
 3. The Seller performs **Work**.
 4. Then **Send Halfway Update**.
 5. The Platform performs **Charge 2nd Third**.
 6. The Seller performs **Complete Order**.
 7. The Platform performs **Charge Final Third**.
 8. Then **Deduct 10% Commission**.
 9. Then **Dispatch Courier**.
 10. **Dispatch Courier** flows up into a final merge node in the Buyer's lane.

Shared Section

Normal Order path, parallel to the custom order's payment/dispatch steps:

1. **Process Payment**, Deduct 15% Commission flows into the Seller's **Confirm & Prepare Item**.
2. **Confirm & Prepare Item** flows into the Platform's **Dispatch Counter API**.
3. **Dispatch Counter API** flows straight down and then into the same **final merge**

4. **node** in the Buyer's lane (joining with the Dispatch Courier path from the custom order branch).

Ending Node:

1. From the final merge node, the Buyer performs **Track**.
2. Then **Confirm Receipt**.
3. Then **Review**.
4. The flow concludes at the **End node**.

Decision Points:

- Order Type → Normal vs. Custom
- Private or Open → visibility of the custom request
- Accept? → Buyer's response to a seller proposal/offer
- Block Seller? → appears only after a rejection.

Merge Points:

- Merge 1: joins **Pick Request** (via Post Publicly) and **Notify Specific Seller** (Private path) → both lead to **Send Proposal**
- Merge 2: joins **Deliver Proposal** to **Buyer** and the renegotiation loop (**Send Offer**, after Block Seller = No) → both lead to **Accept?**
- Merge 3 (final): joins Dispatch Courier (custom order path) and Dispatch **Counter API** (normal order path) → both lead to **Track**

Chapter 4: System Design

4.1 System Architecture

Manzili follows a decoupled, three-tier client-server architecture, where the frontend, backend, and database layers are developed, deployed, and maintained independently. This separation of concerns improves scalability, maintainability, and allows each layer to be updated or scaled without affecting the others.

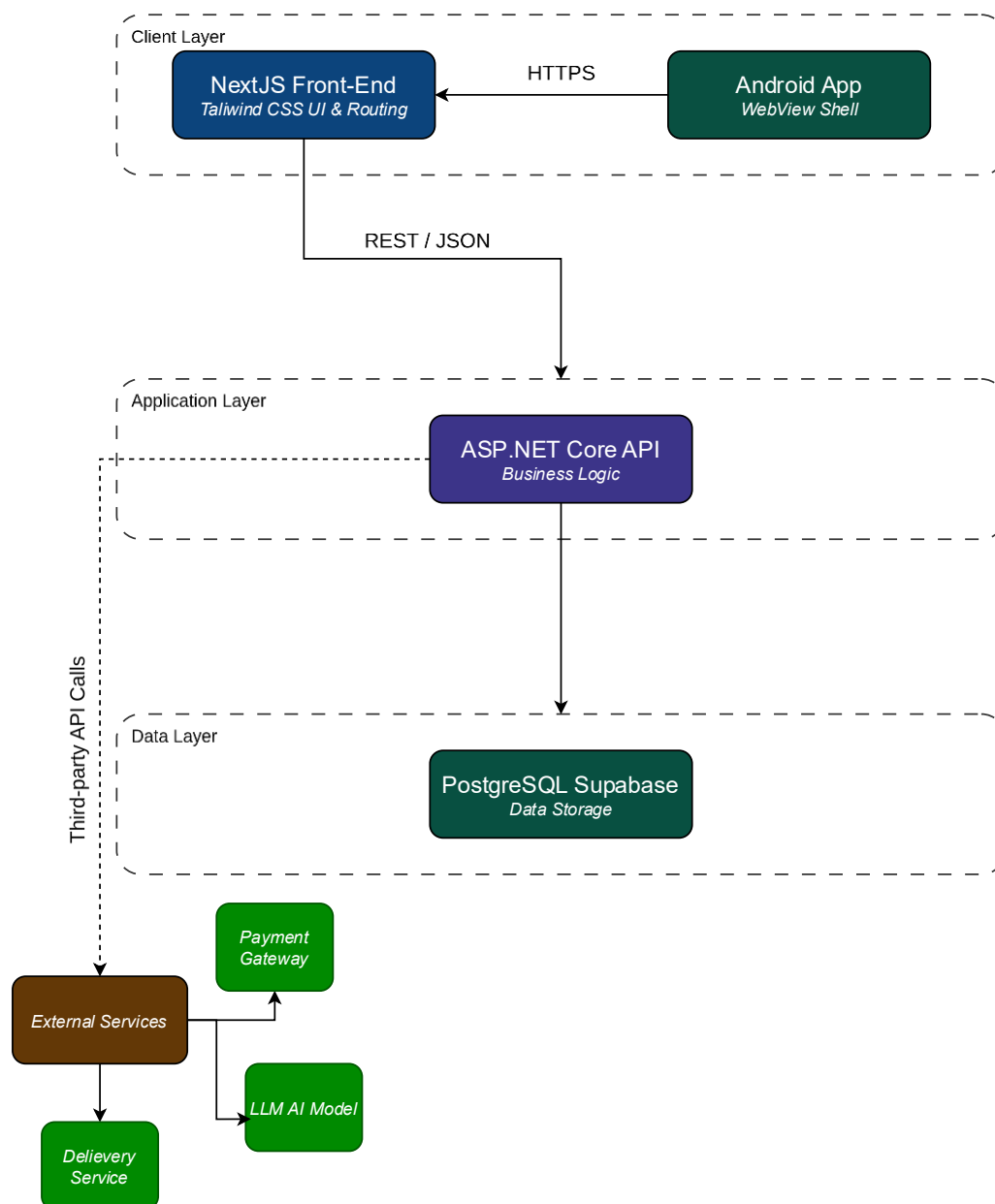


Figure 6: System Architecture Overview

Architecture layers:

- **Presentation Layer:** The user-facing interfaces through which customers, vendors interact with the platform. This includes the Next.js web application and the Android mobile application, which wraps the web application inside a WebView shell to provide a consistent mobile experience without maintaining a separate mobile codebase.
- **Application Layer:** The ASP.NET Core backend exposes a RESTful API that serves as the single entry point for all client applications. This layer handles the platform's core business logic including user authentication, product management, order processing, custom request handling, chat, notifications, and integration with third-party services.
- **Data Layer:** A PostgreSQL database managed through Supabase, which stores all persistent platform data including users, products, orders, messages, and financial records. Supabase also provides authentication support, real-time capabilities for features such as live chat and order status updates, and file storage for product and custom-request media.
- **External Services Layer:** Third-party APIs that extend the platform's core capabilities: Stripe and Kashier for secure payment processing, Bosta for delivery logistics and shipment tracking, Cloudinary for media hosting and image delivery, Jina AI for generating semantic vector embeddings, and Gemini API and DeepSeek V4 for AI-powered features across the platform.

4.2 Database Design (ERD)

The Manzili database follows an Entity-Relationship model designed to support the full lifecycle of a multi-vendor handmade marketplace, including standard shopping, custom product commissioning, in-platform messaging, seller payouts, returns, and coupon management. The complete ERD is provided below as an overview, followed by a breakdown into focused entity groups.

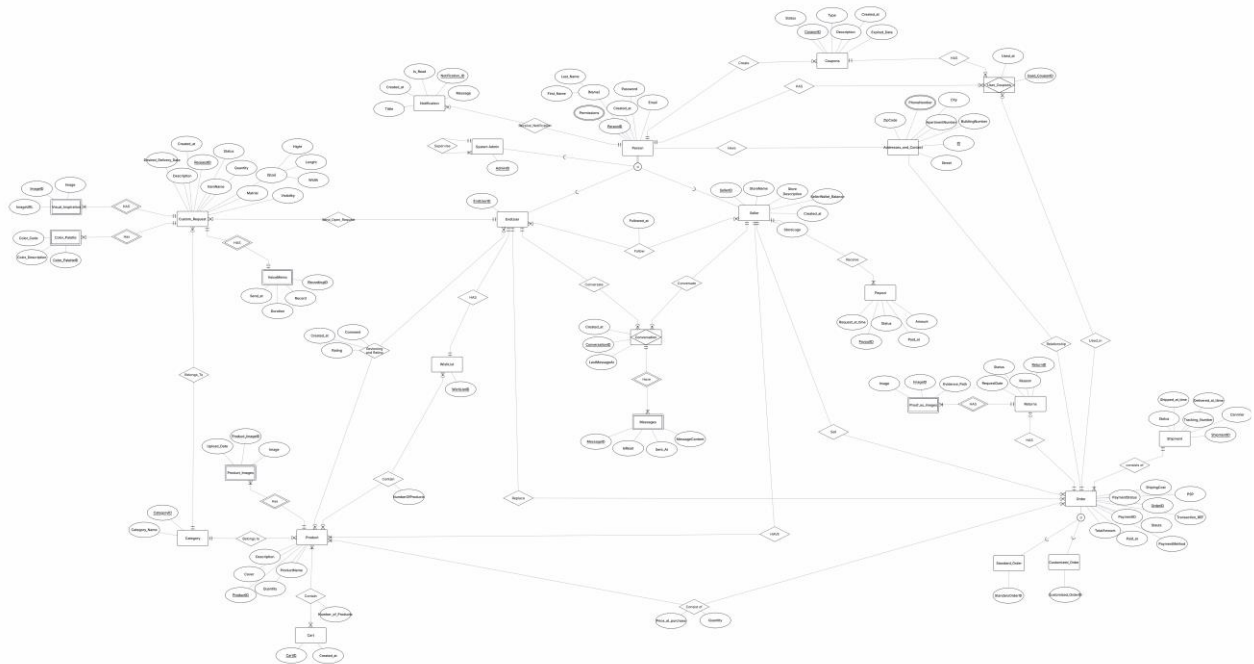


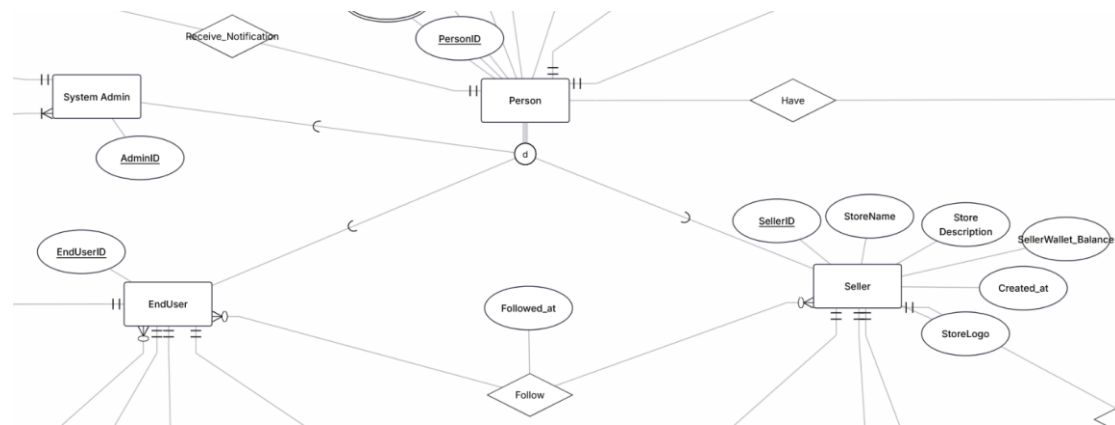
Figure 7: Entity Relationship Diagram

For better ERD visibility click [here](#) or by scanning the QR code



4.2.1 User & Identity Entity Group

This group covers all registered participants on the platform. The design uses a **supertype/subtype (specialization)** pattern, where **Person** is the supertype holding shared identity attributes, and EndUser, Seller, and System Admin are the three subtypes representing the distinct roles a person can hold on the platform.



4.2.1.1 Entities

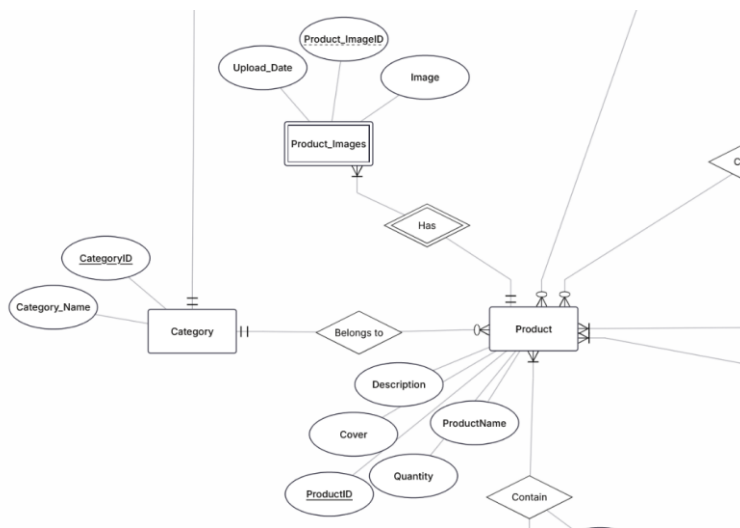
Entity	Description
Person	The Person entity stores shared attributes including PersonID, Email, Password, Name, Permissions, and Created_at. Every registered user on Manzili is a Person before specializing into a role.
End User	Represents customers who browse, shop, and place orders on the marketplace.
Seller	Represents artisans and makers who operate storefronts, holding additional attributes such as StoreName, StoreDescription, StoreLogo, and SellerWallet_Balance.
System Admin	Represents platform administrators, and supports a self-referencing Supervise relationship, meaning one admin can supervise other admins in a hierarchical structure.
Address_and_Contact	Represent Delivery and contact details (ID, City, Street, BuildingNumber, ApartmentNumber, ZipCode, PhoneNumber).
Notifications	Represent System alerts sent to users (Notification_ID, Tittle, Message, Created_at, Is_Read).

4.2.1.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Have	Person → Addresses_and _Contact	1:1	Each Person has exactly one address and contact record for delivery and communication purposes
Receive Notification	Person → Notification	1:M	A Person can receive many notifications for order updates, messages, and platform events
Supervise	System Admin → System Admin	1:M	A System Admin can supervise another System Admins forming an administrative hierarchy
Follow	EndUser → Seller	M:M	An EndUser can follow many Sellers and a Seller can be followed by many EndUsers to stay updated on their stores

4.2.2 Product & Category Entity Group

This group covers the product catalog that sellers maintain on the platform.



4.2.2.1 Entities

Entity	Description
Product	Represents items listed for sale (ProductID, ProductName, Quantity, Description, Cover).

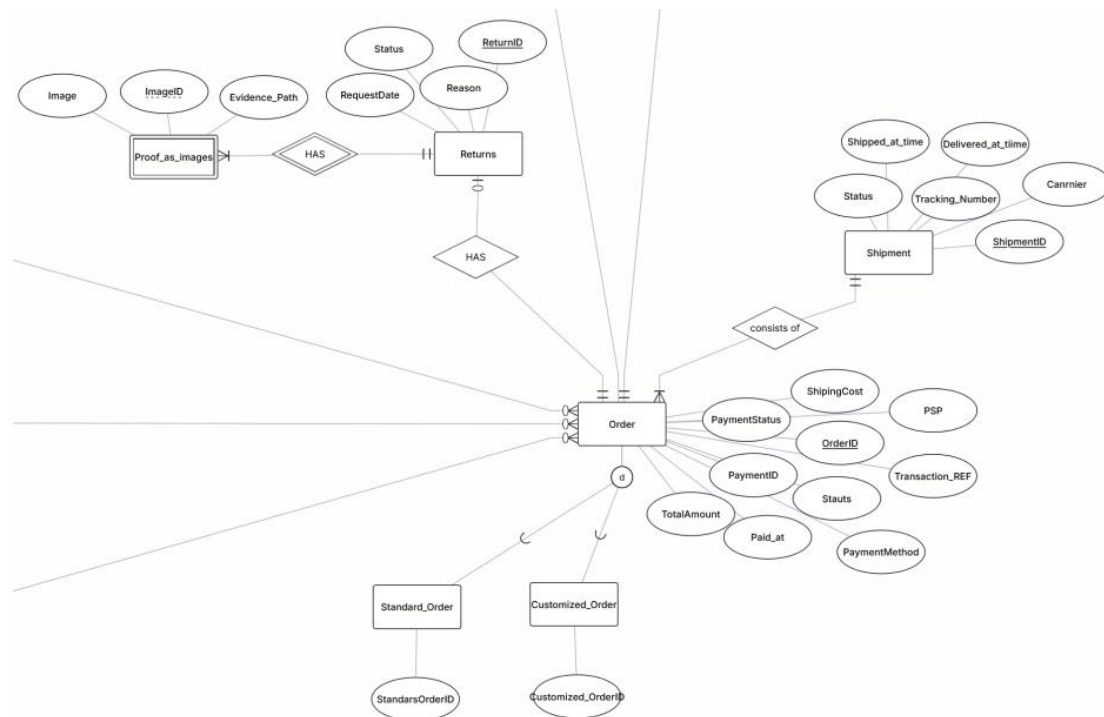
Product_Image	Weak Entity Image assets attached to a Product (Product_ImageID, Image, Upload_Date). Cannot exist without a parent Product.
Category	Categories provide the classification structure used for browsing and filtering handmade goods on the platform.
Cart	Represent Shopping cart holding products before checkout (CartID, Created_at).
Wishlist	Represent a saved list of products for an EndUser.

4.2.2.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Have	Seller → Product	1:M	A Seller can list one or many Products on their storefront, while each Product belongs to exactly one Seller
Has	Product → Product_Images	1:M	A Product has one or many Product_Images attached to it, while each image belongs to exactly one Product
Belongs To	Product → Category	M:1	Many Products can be classified under one Category, while a Category can contain many Products
Has	EndUser → Cart	1:1	Each EndUser has exactly one Cart for staging products before checkout
Has	EndUser → WishList	1:1	Each EndUser has exactly one WishList for saving products of interest
Reviewing and Rating	EndUser → Product	M:M	An EndUser can review and rate many Products and a Product can be reviewed and rated by many EndUsers

4.2.3 Order & Fulfillment Entity Group

This group covers the complete order lifecycle from placement through delivery.



4.2.3.1 Entities

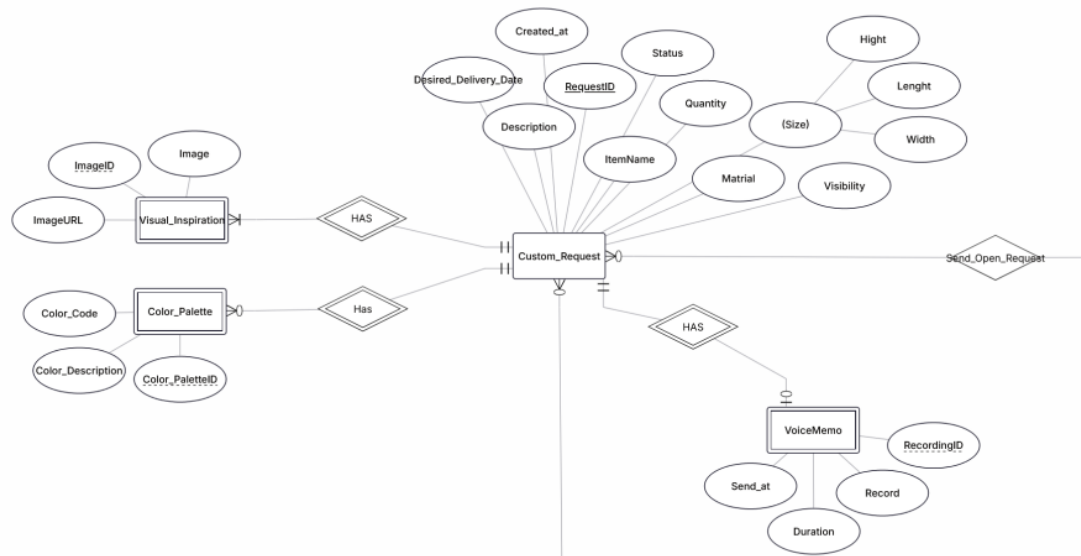
Entity	Description
Order	Supertype entity that captures all purchase transactions, holding attributes including OrderID, Status, ShippingCost, TotalAmount, and full payment details (PaymentID, PaymentMethod, PSP, PaymentStatus, Transaction_REF, Paid_at).
Standard_Order	A regular product purchase order subtype.
Customized_Order	A bespoke or made-to-order purchase subtype.
Shipment	Track carrier, status, shipping and delivery timestamps, and tracking number via integration with the Delivery Service API.
Returns	handles return and refund requests initiated against an order, storing the reason, status, and request date.
Proof_as_images	Weak Entity that stores photo evidence submitted by the customer to support the return claim.

4.2.3.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Place	EndUser → Order	1:M	An EndUser can place one or many Orders while each Order must be placed by exactly one EndUser.
Sell	Seller → Order	1:M	A Seller can fulfill one or many Orders while each Order must be fulfilled by exactly one Seller.
Consist of	Order → Product	M:M	An Order can consist of many Products and a Product can appear in many Orders as line items.
Consists of	Order → Shipment	1:M	An Shipment can have one or many Order for delivery tracking while each Order must belong to exactly one Shipment.
Ships to	Order → Addresses_and _Contact	1:1	Many Orders can be shipped to the same address while each Order is linked to exactly one delivery address.
Has	Order → Returns	1:1	An Order can have at most one Return request initiated against it.
Has	Returns → Proof_as_image	1:M	A Return can be supported by one or many Proof_as_images submitted as evidence while each image belongs to exactly one Return.

4.2.4 Custom Request Entity Group

This group covers Manzili's distinctive custom commissioning feature, which allows EndUsers to post open requests for bespoke handmade items.



4.2.4.1 Entities

Entity	Description
Custom_Request	The Custom_Request entity stores the full specification of a bespoke request: RequestID, ItemName, Description, Visibility, Quantity, Size, Material, Desired_Delivery_Date, Created_at, and Status. Each request is submitted by an EndUser
Visual_Inspiration	Weak Entity Stores reference images the customer uploads to guide the maker, holding ImageID, Image, and ImageURL.
VoiceMemo	Weak Entity Stores spoken instructions recorded by the customer, with attributes RecordingID, Duration, Send_at, and Record.
Color_Palette	Weak entity stores color preferences specified by the customer, with Color_PaletteID, Color_Code, and Color_Description.

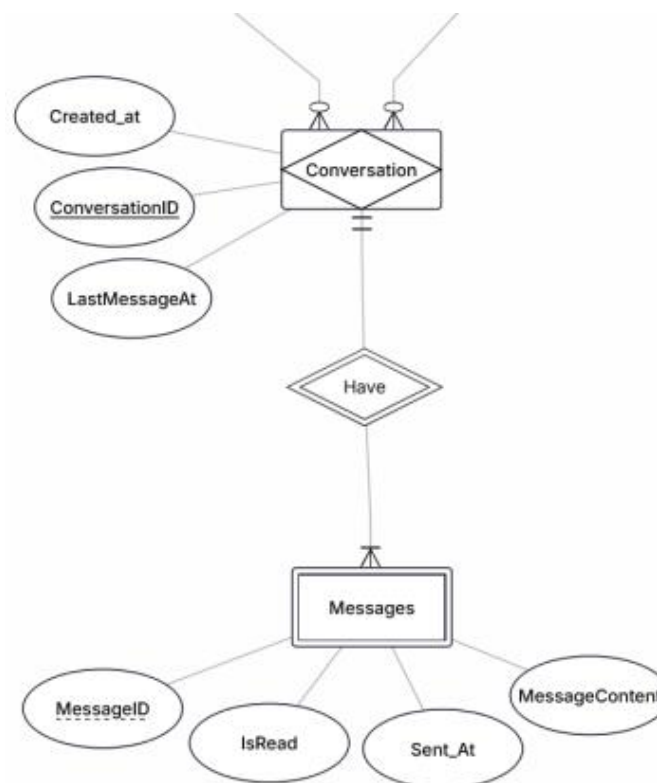
4.2.4.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Send Open Request	EndUser → Custom_Request	1:M	An EndUser can submit one or many Custom_Requests while each Custom_Request must be submitted by exactly one EndUser

Belongs To	Custom_Request → Category	M:1	A Custom_Request can be classified under exactly one Category while a Category can be linked to many Custom_Requests
Has	Custom_Request → Visual_Inspiration	1:M	A Custom_Request can have one or many Visual_Inspiration images attached to guide the maker while each image must belong to exactly one Custom_Request
Has	Custom_Request → VoiceMemo	1:1	A Custom_Request can have at most one VoiceMemo containing spoken instructions from the customer
Has	Custom_Request → Color_Palette	1:M	A Custom_Request can have one or many Color_Palette entries specifying color preferences while each entry must belong to exactly one Custom_Request

4.2.5 Communication Entity Group

This group covers the in-platform messaging system between EndUsers and Sellers.



4.2.5.1 Entities

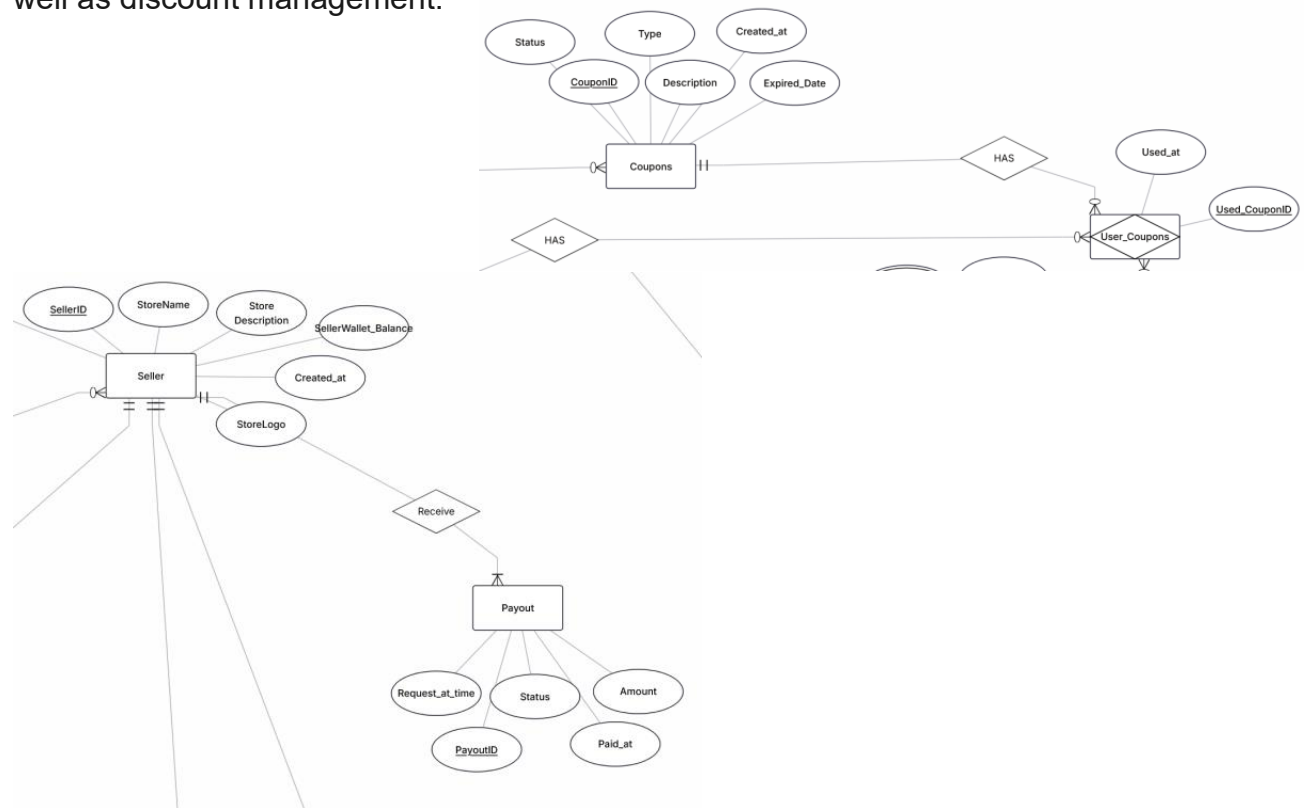
Entity	Description
Conversation	Entity acts as an associative entity linking an EndUser and a Seller into a messaging thread, storing ConversationID, Created_at, and LastMessageAt. A conversation can involve many EndUsers and many Sellers, supporting the platform's direct buyer-seller communication feature.
Messages	Weak Entity dependent on a parent Conversation, storing each individual message exchanged within the thread. Its attributes include MessageID, MessageContent, Sent_At, and IsRead, which tracks whether the recipient has read the message.

4.2.5.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Conversate	EndUser → Conversation	1:M	An EndUser can participate in one or many Conversations while each Conversation must be linked to exactly one EndUser
Conversate	Seller → Conversation	1:M	A Seller can participate in one or many Conversations while each Conversation must be linked to exactly one Seller
Have	Conversation → Messages	1:M	A Conversation can contain one or many Messages while each Message must belong to exactly one Conversation

4.2.6 Financial Operations Entity Group

This group covers the financial flows between the platform and its sellers, as well as discount management.



4.2.6.1 Entities

Entity	Description
Payout	Entity represents seller withdrawal requests from their accumulated wallet balance (SellerWallet_Balance on the Seller entity). Each payout stores PayoutID, Amount, Status, Request_at_time, and Paid_at.
Coupons	Stores discount codes created on the platform, holding CouponID, Description, Created_at, Status, Type, and Expired_Date. Coupons are typically created by authorized Persons (admins) and distributed to users.
User_Coupons	Associative entity tracks the redemption of coupons by specific users on specific orders, storing Used_CouponID and Used_at, and linking a Person, a Coupon, and an Order together.

4.2.6.2 Relationships

Relationship	Entities Involved	Cardinality	Description
Receive	Seller → Payout	1:M	A Seller can receive one or many Payouts from their wallet balance while each Payout must belong to exactly one Seller
Create	Person → Coupons	1:M	A Person (typically an Admin) can create one or many Coupons while each Coupon must be created by exactly one Person
HAS	Coupons → User_Coupons	1:M	A Coupon can be distributed to many Users through User_Coupons records while each User_Coupons record must reference exactly one Coupon
HAS	Person → User_Coupons	1:M	A Person can hold one or many User_Coupons records tracking their coupon redemptions while each record must belong to exactly one Person
Used_in	Conversation → Messages	M:1	A User_Coupon record can be applied to an Order at checkout while each Order can use at most one User_Coupon

4.3 Logical Schema Design

The logical schema defines the relational table structure of the Manzili database, derived directly from the ERD presented in Section 4.2. Each entity is mapped to a table with its attributes, data types, primary keys (PK), and foreign keys (FK). The schema is designed to enforce data integrity, support the platform's business rules, and reflect the relationships identified in the ERD.

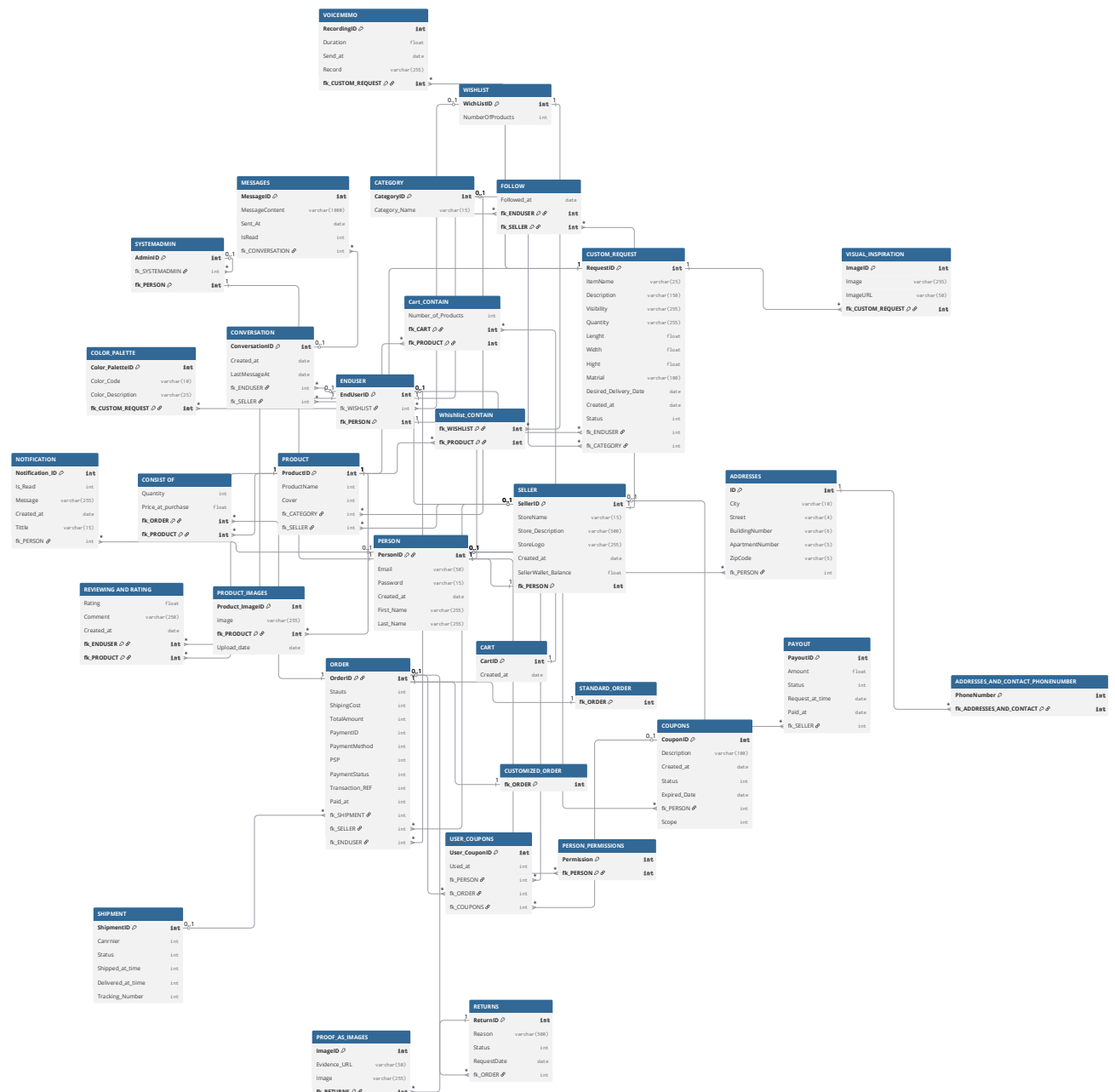


Figure 8: Logical Schema

4.3.1 User & Identity Tables

This group contains the tables responsible for storing all registered user data on the Manzili platform. The design follows a supertype/subtype pattern where the Person table acts as the central identity record and EndUser, Seller, and System_Admin tables extend it with role-specific attributes. Supporting tables handle permissions, addresses, and contact details.

4.3.1.1 Person Table

PERSON	
PersonID	int
Email	varchar(50)
Password	varchar(15)
Created_at	date
First_Name	varchar(255)
Last_Name	varchar(255)

The Person table is the supertype table that stores shared identity attributes for every registered user on the platform regardless of their role. All subtype tables reference this table through a one-to-one foreign key relationship.

4.3.1.2 Person_Permissions Table

PERSON_PERMISSIONS	
Permission	int
fk_PERSON	int

The Person_Permissions table stores role-based permission flags assigned to a Person. It is modeled as a separate table to support multiple permissions per user, following a normalized design.

SELLER	
SellerID	int
StoreName	varchar(15)
Store_Description	varchar(500)
StoreLogo	varchar(255)
Created_at	date
SellerWallet_Balance	float
fk_PERSON	int

4.3.1.3 EndUser Table

The EndUser table extends the Person table to represent customers on the platform. It holds a direct reference to the Person supertype and links to the user's WishList.

4.3.1.4 Seller Table

SELLER	
SellerID	int
StoreName	varchar(15)
Store_Description	varchar(500)
StoreLogo	varchar(255)
Created_at	date
SellerWallet_Balance	float
fk_PERSON	int

The Seller table extends the Person table to represent artisans and makers who operate storefronts on the platform. It stores storefront-specific attributes including store branding and wallet balance.

4.3.1.5 System_Admin Table

SYSTEMADMIN	
AdminID	int
fk_SYSTEMADMIN	int
fk_PERSON	int

The System_Admin table extends the Person table to represent platform administrators. It includes a self-referencing foreign key to support the administrative hierarchy where one admin can supervise another.

4.3.1.6 Addresses Table

ADDRESSES	
ID	int
City	varchar(10)
Street	varchar(4)
BuildingNumber	varchar(6)
ApartmentNumber	varchar(5)
ZipCode	varchar(5)
fk_PERSON	int

The Addresses table stores delivery address details associated with a Person. Each Person record is linked to one address record used for order shipping purposes.

4.3.2.7 Addresses_and_Contact_PhoneNumber Table

ADDRESSES_AND_CONTACT_PHONENUMBER	
PhoneNumber	int
fk_ADDRESSES_AND_CONTACT	int

The Addresses_and_Contact_PhoneNumber table stores phone numbers linked to an address record. It is modeled as a separate table to support multiple phone numbers per address in a normalized structure.

4.3.1.8 Notification Table

NOTIFICATION	
Notification_ID	int
Is_Read	int
Message	varchar(255)
Created_at	date
Tittle	varchar(15)
fk_PERSON	int

The Notification table stores system alerts sent to any Person on the platform, including order updates, new messages, and platform events.

4.3.1.9 Follow Table

FOLLOW	
Followed_at	date
fk_ENDUSER	int
fk_SELLER	int

The FOLLOW table is an associative table that tracks the follow relationship between EndUsers and Sellers, recording when a customer chose to follow a specific seller's storefront.

4.3.2 Product & Category Tables

This group contains the tables responsible for managing the product catalog on the Manzili platform. It covers product listings, image assets, classification categories, and the two browsing-related features — the wishlist and shopping cart.

4.3.2.1 Category Table

CATEGORY	
CategoryID	int
Category_Name	varchar(15)

The category table provides the classification structure used for browsing and filtering handmade goods across the platform. Each product and custom request is linked to exactly one

category, making this table a central reference point for product organization.

4.3.2.2 Product Table

PRODUCT	
ProductID	int
ProductName	int
Cover	int
fk_CATEGORY	int
fk_SELLER	int

The PRODUCT table stores core listing information for each item listed by a seller, including the product name, cover image, and references to both the owning Seller and the associated Category. It serves as the central entity in this group, with several other tables depending on it directly.

4.3.2.3 Product_Images Table

PRODUCT_IMAGES	
Product_ImageID	int
image	varchar(255)
fk_PRODUCT	int
Upload_date	date

The PRODUCT_IMAGES table is a weak entity table that stores additional image assets attached to a product beyond the cover photo. It uses a composite primary key combining its own image ID with the foreign key to the parent Product, reflecting its dependency on the parent record and ensuring images cannot exist without an associated product.

4.3.2.4 Wishlist Table

WISHLIST	
WichListID	int
NumberOfProducts	int

The WISHLIST table represents each EndUser's personal saved products list. It maintains a one-to-one relationship with the ENDUSER table, meaning every EndUser has exactly one WishList on the platform.

4.3.2.5 Wishlist_Contain Table

Whishlist_CONTAIN	
fk_WISHLIST	int
fk_PRODUCT	int

The Wishlist_CONTAIN table is an associative table that resolves the many-to-many relationship between WishLists and Products. It uses a composite primary key combining the WishList and Product foreign keys, recording which products a specific user has saved for later.

4.3.2.6 Cart Table

CART	
CartID	int
Created_at	date

The CART table represents the EndUser's active shopping cart, storing a reference to its creation date. Similar to the WishList, it maintains a relationship with the ENDUSER table as the cart's owner.

4.3.2.7 Cart_Contain Table

Cart_CONTAIN	
Number_of_Products	int
fk_CART	int
fk_PRODUCT	int

The Cart_CONTAIN table is an associative table that resolves the many-to-many relationship between Carts and Products. Beyond the composite primary key, it additionally stores the quantity of each product added to the cart, which is a critical attribute for calculating order totals at checkout.

4.3.2.8 Reviewing And Rating Table

REVIEWING AND RATING	
Rating	float
Comment	varchar(250)
Created_at	date
fk_ENDUSER	int
fk_PRODUCT	int

The REVIEWING AND RATING table stores customer reviews and ratings submitted for products on the platform. It uses a composite primary key combining the EndUser and Product foreign keys, enforcing that each customer can submit only one review per product. It stores a numeric Rating, an optional written Comment, and a Created_at timestamp recording when the review was submitted.

4.3.3 Order & Fulfillment Tables

This group covers the complete order lifecycle on the Manzili platform, from order placement and payment through delivery tracking and returns.

4.3.3.1 Order Table

ORDER	
OrderID 🔗 🔗	int
Stauts	int
ShippingCost	int
TotalAmount	int
PaymentID	int
PaymentMethod	int
PSP	int
PaymentStatus	int
Transaction_REF	int
Paid_at	int
fk_SHIPMENT 🔗	int
fk_SELLER 🔗	int
fk_ENDUSER 🔗	int

The Order table is the central entity in this group, storing the full transaction record for every purchase made on the platform. It consolidates payment-related attributes directly — including PaymentID, PaymentMethod, PSP, PaymentStatus, Transaction_REF, and Paid_at — since each order has exactly one payment transaction, making a separate payment table unnecessary. It also references the placing EndUser, the fulfilling Seller, and the associated Shipment.

4.3.3.2 Standrad_Order Table

STANDARD_ORDER	
fk_ORDER 🔗	int

The STANDARD_ORDER table is a subtype table that extends the ORDER table in a one-to-one relationship, representing a regular catalog purchase transaction. It follows the same supertype/subtype pattern used in the user identity group, inheriting all shared order attributes from the parent ORDER table.

4.3.3.3 Customized_Order

CUSTOMIZED_ORDER	
fk_ORDER 🔗	int

The CUSTOMIZED_ORDER table is a subtype table that extends the ORDER table in a one-to-one relationship, representing a bespoke commissioned purchase transaction. Like STANDARD_ORDER, it

inherits all shared attributes from the parent ORDER table while marking the order as a custom commission.

4.3.3.4 Consist of (Order Item) Table

CONSIST OF	
Quantity	int
Price_at_purchase	float
fk_ORDER	int
fk_PRODUCT	int

The CONSIST OF table is an associative table that resolves the many-to-many relationship between Orders and Products, representing the individual line items within each order. It stores the quantity and the price at the time of purchase for each product. Recording the price at purchase rather than referencing the current product price is a critical design decision — it ensures order history remains accurate even if the product price is updated by the seller at a later date.

4.3.3.5 Shipment Table

SHIPMENT	
ShipmentID	int
Carrier	int
Status	int
Shipped_at_time	int
Delivered_at_time	int
Tracking_Number	int

The SHIPMENT table tracks the delivery of an order, storing carrier information, current status, tracking number, and timestamps for both shipment dispatch and delivery completion. It integrates with the Bosta delivery API to generate and update shipment records throughout the delivery lifecycle.

4.3.3.6 Returns Table

RETURNS	
ReturnID	int
Reason	varchar(500)
Status	int
RequestDate	date
fk_ORDER	int

The RETURNS table handles return and refund requests initiated by customers against a placed order. It stores the customer's stated reason, the current return status, and the date the request was submitted, linked directly to the associated ORDER record.

4.3.3.7 Proof_As_Images Table

PROOF_AS_IMAGES	
ImageID	int
Evidence_URL	varchar(50)
Image	varchar(255)
fk_RETURNS	int

The PROOF_AS_IMAGES table is a weak entity table that stores photo evidence submitted by the customer to support a return request. It uses a composite primary key combining its own image ID with the foreign key to the parent RETURNS record, reflecting its dependency on the return request and ensuring proof images cannot exist without an associated return.

4.3.4 Custom Request Tables

This group covers Manzili's distinctive custom commissioning feature, which allows EndUsers to post open requests for bespoke handmade items directly on the platform.

4.3.4.1 Custom_Request Table

CUSTOM_REQUEST	
RequestID	int
ItemName	varchar(25)
Description	varchar(150)
Visibility	varchar(255)
Quantity	varchar(255)
Lenght	float
Width	float
Hight	float
Matrial	varchar(100)
Desired_Delivery_Date	date
Created_at	date
Status	int
fk_ENDUSER	int
fk_CATEGORY	int

The CUSTOM_REQUEST table stores the full specification of a bespoke request submitted by an EndUser. Beyond basic descriptive fields such as item name, description, and material, it stores physical dimension attributes — Length, Width, and Height — allowing customers to specify exact size requirements for their commissioned item. A Visibility attribute controls whether the request is publicly visible to all sellers on the platform or restricted to a specific seller. The table references both the submitting EndUser and the relevant Category, and tracks the request lifecycle through a Status attribute.

4.3.4.2 Visual_Inspiration Table

VISUAL_INSPIRATION	
ImageID	int
Image	varchar(255)
ImageURL	varchar(50)
fk_CUSTOM_REQUEST	int

The VISUAL_INSPIRATION table is a weak entity table that stores reference images uploaded by the customer to visually guide the maker during production. It uses a composite primary key combining its own image ID with the foreign key to the parent Custom Request, reflecting its dependency on the parent record.

4.3.4.3 Voicemail Table

VOICEMEMO	
RecordingID	int
Duration	float
Send_at	date
Record	varchar(255)
fk_CUSTOM_REQUEST	int

The VOICEMEMO table is a weak entity table that stores spoken instructions recorded by the customer as an audio note attached to a custom request. It records the duration of the recording, the submission timestamp, and a reference to the stored audio file. Like other weak entities in this group, it uses a composite primary key combining its own recording ID with the foreign key to the parent Custom Request.

4.3.4.4 Color_Palette Table

COLOR_PALETTE	
Color_PaletteID	int
Color_Code	varchar(10)
Color_Description	varchar(25)
fk_CUSTOM_REQUEST	int

The COLOR_PALETTE table is a weak entity table that stores specific color preferences submitted by the customer as part of a custom request. Each entry holds a color code and an optional description, allowing customers to communicate precise color requirements to the maker. It also uses a composite primary key combining its own palette ID with the foreign key to the parent Custom Request.

4.3.5 Communication Tables

This group covers the in-platform messaging system that enables direct communication between EndUsers and Sellers on the Manzili platform.

4.3.5.1 Conversation Table

CONVERSATION	
ConversationID 	int
Created_at	date
LastMessageAt	date
fk_ENDUSER 	int
fk_SELLER 	int

The CONVERSATION table acts as an associative entity linking exactly one EndUser and one Seller into a persistent messaging thread. It stores the creation date of the conversation and a LastMessageAt timestamp, which records the time of the most recent message sent within the thread. The LastMessageAt attribute is particularly useful for sorting conversations by recent activity and displaying message previews in the chat interface without querying the MESSAGES table directly.

4.3.5.2 Messages Table

MESSAGES	
MessageID 	int
MessageContent	varchar(1000)
Sent_At	date
IsRead	int
fk_CONVERSATION 	int

The MESSAGES table stores each individual message exchanged within a conversation, recording the full message content, the timestamp it was sent, and a read status flag through the IsRead attribute. The IsRead attribute enables unread message indicators in the chat interface, allowing both EndUsers and Sellers to identify messages that have not yet been read. Each message belongs to exactly one Conversation through a foreign key reference, reflecting the weak entity nature of messages relative to their parent conversation.

4.3.6 Financial Operations Tables

This group covers the financial flows between the platform and its sellers, as well as the discount and coupon management system available to users on the Manzili platform.

4.3.6.1 Payout Table

PAYOUT	
PayoutID 🔗	int
Amount	float
Status	int
Request_at_time	date
Paid_at	date
fk_SELLER 🔗	int

The PAYOUT table represents seller withdrawal requests from their accumulated earnings stored as SellerWallet_Balance in the SELLER table. Each payout record tracks the requested amount, the current status of the payout, and two timestamps — one for when the request was submitted and one for when the payout was completed. A Seller can have many payout records over time, building a full financial history of their withdrawals from the platform.

4.3.6.2 Coupons Table

COUPONS	
CouponID 🔗	int
Description	varchar(100)
Created_at	date
Status	int
Expired_Date	date
fk_PERSON 🔗	int
Scope	int

The COUPONS table stores discount codes created on the platform by authorized Persons, typically administrators. Beyond standard attributes such as description, creation date, status, and expiry date, it includes a Scope attribute that defines the applicability of the coupon — for example whether it applies platform-wide or is restricted to specific products or categories. Each coupon references the Person who created it, maintaining accountability for coupon issuance on the platform.

4.3.6.3 User_Coupons Table

USER_COUPONS	
User_CouponID 🔗	int
Used_at	int
fk_PERSON 🔗	int
fk_ORDER 🔗	int
fk_COUPONS 🔗	int

The USER_COUPONS table is a three-way associative table that links a Person, a Coupon, and an Order together in a single record. It tracks when a specific coupon was redeemed by a specific user on a specific order through the Used_at timestamp. This three-way relationship design ensures full auditability of coupon usage across the platform, preventing duplicate redemptions and providing a clear record of every discount applied to every order.

4.3.7 Business Process Flow (High-Level)

The following describes the end-to-end business process implied by the Manzili database schema, tracing the journey of both EndUsers and Sellers across the platform's core operations.

Registration & Identity

Every participant on the platform registers as a Person and specializes into one of three roles — EndUser, Seller, or System Admin. Upon registration, each Person is assigned an address record and a permissions record. Sellers additionally set up their storefront profile before they can list products.

Product Discovery

Once registered, EndUsers can browse and search products organized by Category. They can save products of interest to their WishList or add them directly to their Cart. EndUsers can also follow Sellers to stay updated on their stores, and view reviews and ratings submitted by other customers on any product.

Standard Order Placement

When an EndUser is ready to purchase, they proceed from their Cart to checkout, where a Standard_Order is created under the parent Order record. Payment is processed through Stripe and captured directly on the Order. A Shipment record is then generated via the Bosta API to handle delivery to the EndUser's registered address. Coupons can be applied at checkout and are tracked through the User_Coupons record linked to the Order.

Custom Commissioning

Alternatively, an EndUser can submit a Custom_Request specifying the item name, dimensions, material, desired delivery date, and visibility. They can enrich the request with Visual_Inspiration images, a VoiceMemo, and Color_Palette entries to communicate their vision clearly to makers. Once a Seller accepts the request, a Customized_Order is created to fulfill it.

Communication

Throughout the buying process, EndUsers and Sellers can communicate directly through the Conversation and Messages tables, supporting pre-sale inquiries, custom request discussions, and post-sale support.

Order Fulfillment & Returns

The Seller updates the Order and Shipment status as the item progresses through production and delivery. If the EndUser is unsatisfied, they can initiate a Return against the Order, submitting Proof_as_images as evidence to support their claim.

Financial Operations

As orders are completed, Sellers accumulate earnings in their SellerWallet_Balance. They can request Payouts at any time, which are tracked through the Payout table. The platform also manages Notifications to keep all Persons informed of relevant events throughout the entire process.

4.3.8 What This Schema Represents

- This schema models Manzili as a multi-vendor e-commerce marketplace that supports:
- Standard marketplace operations: Product listings, shopping carts, wish lists, orders, shipments, and seller payouts.
- Custom/bespoke commerce: Open custom requests with rich media attachments (images, voice memos, color palettes), enabling made-to-order products beyond the standard catalog.
- Social marketplace features: Seller following, product reviews, and in-platform messaging between buyers and sellers.
- Promotional tooling: Coupon creation, distribution, and redemption tracking tied to specific orders and users.
- Post-purchase support: Returns with photo evidence and a hierarchical admin structure for platform governance.

For better Schema visibility you access its interactive version through [here](#)

or by scanning the QR code



4.4 API Design

The Manzili backend exposes a RESTful API that serves as the single communication channel between the presentation layer and the application logic.

The API was designed collaboratively by the backend and frontend team members using Apidog, an API design and documentation tool. This collaborative approach ensured that endpoint contracts including request formats, response structures, authentication requirements, and error codes were agreed upon before implementation.

The interactive API documentation, which includes all endpoint definitions, request/response examples, and authentication flows, is publicly accessible from [here](#) or by scanning the QR code



4.5 UI/UX Wireframes

The UI/UX design phase for Manzili followed a rapid approach focused on aligning the team on layout and user flows before moving into full frontend implementation. Rather than pixel-perfect wireframes, the team prioritized speed and shared understanding.

The process began with competitor and EC templates analysis — reviewing existing e-commerce and handmade marketplace platforms (including imakethis) to identify effective conventions for product browsing, vendor storefronts, checkout flows, and dashboards.

This fed into brief collaborative whiteboard sessions where the team sketched very basic hand drawn page layouts, and key flows like the custom order flow. To accelerate internal alignment, rough AI-generated images were used as visual aids, not as final designs, but as quick shared references for layout direction and component placement.

Here is a recored discussion video link of the very initial concept of the custom order flow from [here](#) or by scanning the QR code,



Chapter 5: Implementation

5.1 Development Environment

Manzili was developed as three independently runnable components — the frontend web application, the backend API, and the cloud database — connected over a REST interface. The frontend never accesses the database directly; all data passes through the backend, which is the only component that holds the database credentials.

Component	Technology	Local Endpoint
Frontend	Next.js 16 (React 19), Tailwind CSS 4, on Node.js	http://localhost:3000
Backend	ASP.NET Core Web API (.NET 10, C#)	http://localhost:5080
Database	PostgreSQL 17 on Supabase (schema: manzili)	Supabase pooler

Development was carried out in Visual Studio 2026 and Visual Studio Code, using Git for version control and Apidog for designing and testing API endpoints. AI-assisted coding tools were used to accelerate routine implementation tasks, with all generated code reviewed and adapted by the team before use. Uploaded media files are hosted on Cloudinary rather than on the application servers, keeping the backend lightweight and storage concerns separate from application logic.

5.2 Frontend Implementation

The frontend is a single Next.js application built with the App Router, in which each route corresponds to a folder under the `app/` directory. The routes are organized into three groups that map directly onto the platform's three user audiences:

- `app/(public)` — the buyer-facing storefront, covering the home page, shop and vendor storefronts, product pages, cart and checkout, orders, wishlist, user profile, the custom request flow, and static informational pages.
- `app/store` — the vendor dashboard, covering product management, orders, custom orders, coupons, warehouses, wallet, returns, and store settings.
- `app/admin` — the administration panel, covering store approval, product moderation, orders, custom requests, reports, coupons, conversations, and moderator management.

Application state — including the cart, authenticated session, wishlist, and notifications — is managed using Redux Toolkit, while the database remains the authoritative source of truth. Each screen loads its data from the backend on mount and re-reads it after any mutating action. All API requests pass through a single API client module (`lib/api/`), which carries the authenticated session held in secure `httpOnly` cookies, unwraps the standard `{ success, data, meta }` response envelope, and automatically refreshes an expired access token before retrying the original request.

The interface is fully bilingual, supporting both English and Arabic. Translations are maintained in two dictionaries (`en.js` and `ar.js`), enabling the platform to serve both language audiences from the same codebase. Styling is implemented entirely with Tailwind CSS utility classes, producing a single responsive design that serves desktop, tablet, mobile browser, and the Android WebView shell without any platform-specific code.

5.3 Backend Implementation

The backend is an ASP.NET Core Web API organized as a layered .NET solution consisting of four projects, each with a single, clearly defined responsibility:

- `Manzili.Api` — controllers, request routing, authentication middleware, and the response envelope and error-handling conventions shared across all endpoints.
- `Manzili.Application` — the business logic services covering checkout, fulfillment, offers, wallet, coupons, returns, and admin operations, along with their Data Transfer Objects (DTOs) and validators.
- `Manzili.Infrastructure` — the database context and all integrations with external services.
- `Manzili.Domain` — the core data entities and status enumerations for orders, payments, shipments, store status, and related domain concepts.

Each domain area is exposed through a dedicated controller whose endpoints are consumed by the frontend. Incoming request data is validated using `FluentValidation`, with validation failures returned as a consistent, structured error envelope. Passwords are hashed with `BCrypt`, and authentication is implemented using JSON Web Tokens — a short-lived access token paired with a longer-lived refresh token, both delivered as secure `httpOnly` cookies. Role-based access control is enforced on every protected endpoint. The administrator role is further subdivided into full admins and moderators, where a moderator is granted access only to the specific sections explicitly assigned to them, enforced server-side rather than relying on client-side restrictions.

Two areas of the backend implementation required particular care. The first is the checkout process, which splits a multi-vendor cart into one sub-order per seller while charging the buyer in a single transaction, recording each line item's price and product details as a snapshot at the time of purchase. The second is the handling of financial operations, which are designed to be safe against retries and duplicate processing: the seller wallet is implemented as an append-only ledger keyed by an idempotency value, ensuring a payment can never be double-counted, and incoming delivery webhooks from Bosta are de-duplicated by payload hash so the same event is never processed twice.

5.4 Database Implementation

database is a PostgreSQL 17 instance hosted on Supabase, with all platform data held in a dedicated schema named manzili. The backend accesses it through Entity Framework Core in a database-first configuration, meaning the live database is the master and the C# entity classes are generated to match it exactly. This approach keeps the running schema as the single source of truth rather than a code-defined model that could drift from the actual database over time.

The schema implements the entity groups defined in Chapter 4 — identity, catalog, shopping, orders, custom requests, communication, fulfillment, and financial operations — with primary keys, foreign keys, and explicit composite keys enforcing referential integrity throughout. In line with the design decisions described in Chapter 4, monetary values are stored as fixed-precision decimal types, line items are snapshotted at purchase time to preserve pricing accuracy, and status fields use dedicated enumerated values for consistency and type safety. Media files are not stored in the database; only their Cloudinary URLs are persisted, which keeps the database compact and avoids storing binary data in relational tables.

5.5 Active Database vs. the ERD Design

The ERD presented in Chapter 4 was produced at the design stage as a conceptual model of the domain, before implementation began. During development, the model was refined to meet the requirements of a real multi-vendor marketplace and live third-party integrations. The running schema therefore differs from the original diagram in several respects, described below.

Naming and Casing

the conceptual diagram used singular, upper-case table names with `fk_`-prefixed foreign keys. The live database follows standard PostgreSQL conventions: lower-case snake_case naming, pluralized where natural (e.g. products, orders), and plain foreign key column names (e.g. personid). Some of these changes were mandatory rather than stylistic — for example, ORDER is a reserved SQL keyword and cannot be used as a table name directly.

The Order Model Became Three-Tier

this is the most significant structural change from the conceptual design. The original ORDER entity modeled a single buyer, a single seller, and one set of totals in a single row, which cannot represent a cart containing items from multiple artisans paid for in one transaction. The live schema addresses this by splitting the order into three tiers:

- orders — the buyer-facing checkout and payment record, whose seller link is now nullable to reflect the multi-vendor nature of a single transaction.
- store_orders — one sub-order per seller, each with its own financial breakdown and fulfillment status.
- store_order_items — the individual line items under each sub-order.

The original standard_order and customized_order tables were retained as thin type markers, as their substantive content moved into the new three-tier structure.

Tables Added for Real Integrations

Several features required database structures that the conceptual ERD did not anticipate. Custom order negotiation introduced offers (a seller's bid with a milestone timeline), offer_messages, and offer_payments. The seller balance model grew from a single column into a full financial ledger, comprising seller_wallet and an append-only wallet_transactions table alongside the retained payout table. Bosta delivery integration added shipment_events, store_pickup_addresses (representing seller warehouses), and bosta_address_mapping. The product catalog was extended with product_variants and variant_options, and content moderation added a reports table.

Type and Width Corrections

The conceptual diagram carried several placeholder data types and unrealistic field widths inherited from the early design phase. The live schema corrects all of these: monetary values are stored as fixed-precision decimal, date and time fields use timestamp, boolean flags use the boolean type, and all text columns are sized to realistic, practical widths appropriate for their content.

5.6 AI Features Implementation

- AI Image Generation: On the custom-order form, the buyer describes a one-of-a-kind item and gets a generated preview image of it. It runs a two-step chain: a multi-modal LLM first turns the form (item, materials, colours, style, voice, other images) into a single optimized image-generation prompt, then an image LLM renders it. This gives buyers

and artisans a shared visual reference before any work or payment begins.

- **AI Price Estimator:** On the custom-order form, it suggests a fair price range in EGP for the request. Rather than guessing, it pulls real prices of similar items in the same category from the live catalog, computes min/median/max statistics, and feeds those concrete comparables plus the item's details to the LLM, which returns a grounded low–high range with a one-line rationale and a sanity floor.
- **AI Input Review:** Before a buyer submits a custom request, this reviews their form and suggests 3–5 targeted clarifying questions (e.g. "What size/material exactly?", "What kind of glossy finish?") so the artisan receives enough detail to quote accurately. The LLM reads the structured form and returns a short JSON list of the most useful missing clarifications, reducing back-and-forth.
- **AI Search ("describe-it"):** Lets shoppers search by meaning instead of keywords ("Mother's day gift that not too expensive", "a cute handbag with a flower on it"). Every product is embedded once into a vector stored in pgvector; at search time the query is embedded and the catalog is ranked by cosine similarity directly in the database. This makes search semantic, multilingual and feels like chatting more than searching, costs ~20 tokens per query regardless of catalog size.
- **AI Recommendations ("For You"):** Personalized product rails on the home page and product pages. The engine gathers signals the buyer already generates; their city, the categories they engage with (orders/cart/wishlist), and the prices they tend to pay; and the LLM selects the best items from the live catalog, reasoning about proximity (seller city vs buyer city), taste (favoured categories), and budget vs quality, returning a short reason per pick.

5.7 System Walkthrough

This section provides a screen-by-screen walkthrough of the Manzili platform as implemented, covering the buyer-facing public site, the vendor dashboard, and the administration panel. All interfaces are fully bilingual, supporting both English and Arabic.

5.7.1 Global Layout

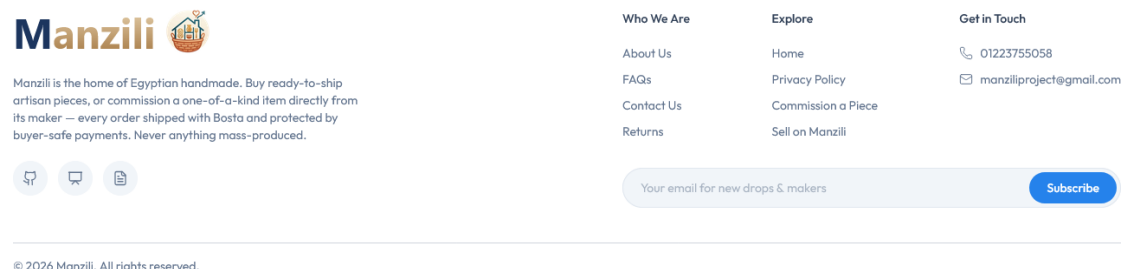
The header and footer described below wrap every page of the public buyer site and are shared across all public-facing screens. The vendor and admin dashboards replace the header with their own sidebars, described in Sections 5.5.4 and 5.5.5 respectively.

Header (Navbar)



- **Description:** The buyer's workspace for a commission, covering its whole lifecycle.
- **Components:** The full request specification (details, images, colours, voice note); the seller's offer; the milestone payment steps (paid by card via Stripe or by mobile wallet via Kashier, with a delivery address captured at the final milestone). Once the final milestone is paid, a delivery-tracking timeline appears; after delivery, the buyer is prompted to leave a review.
- **Navigation:** Milestone payments redirect to the payment provider and return here; a completed, reviewed piece is added to the store's "Custom Work" showcase.

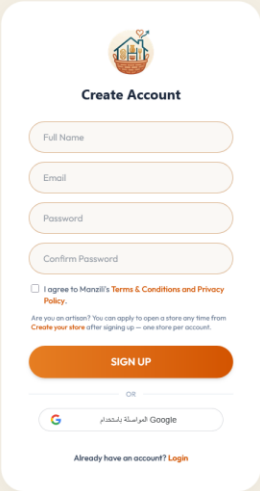
Footer



- **Description:** The site footer present on every public page.
- **Components:** Manzili logo and tagline; quick-link groups "Who We Are" (About, FAQs, Contact, Returns) and "Website" (Home, Privacy Policy, Request a Bespoke Item, Create Your Store); a Contact block (phone and email); external links to the project's GitHub, overview presentation, and documentation; a newsletter subscribe form; and the copyright line.
- **Navigation:** Internal links go to their pages; the GitHub, presentation, and documentation icons open external resources in a new tab; subscribing posts the email to the backend newsletter service.

5.7.2 Authentication

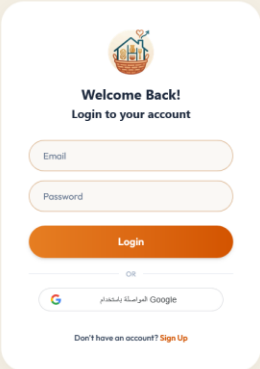
Register (/register)



The register form is titled "Create Account" and features a logo at the top. It includes input fields for "Full Name", "Email", "Password", and "Confirm Password". Below these fields is a checkbox for "I agree to Manzili's Terms & Conditions and Privacy Policy". A note states: "Are you an artisan? You can apply to open a store any time from Create your store after signing up - view store page account". A prominent orange "SIGN UP" button is located below the checkbox. Below the button is a separator line with "OR" in the center, followed by a "Continue with Google" button featuring the Google logo and Arabic text. At the bottom, a link reads "Already have an account? Login".

- Description: Creates a new customer account.
- Components: Name, email, and password fields; "Continue with Google" sign-up; link to the Login page.
- Navigation: On success, logs the user in and returns to the home page. Links to /login.

Login (/login)

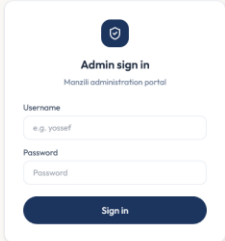


The login form is titled "Welcome Back! Login to your account" and features the same logo as the register form. It includes input fields for "Email" and "Password". A prominent orange "Login" button is located below the password field. Below the button is a separator line with "OR" in the center, followed by a "Continue with Google" button featuring the Google logo and Arabic text. At the bottom, a link reads "Don't have an account? Sign Up".

- Description: Lets existing users sign in to Manzili.
- Components: Email and password fields; "Login" button; "Continue with Google" sign-in; link to the Register page; forgot-password link.

- Navigation: On success, returns the user to where they were (or the home page); sellers gain access to the Store dashboard. Links to /register.

Admin Login (/admin/login)



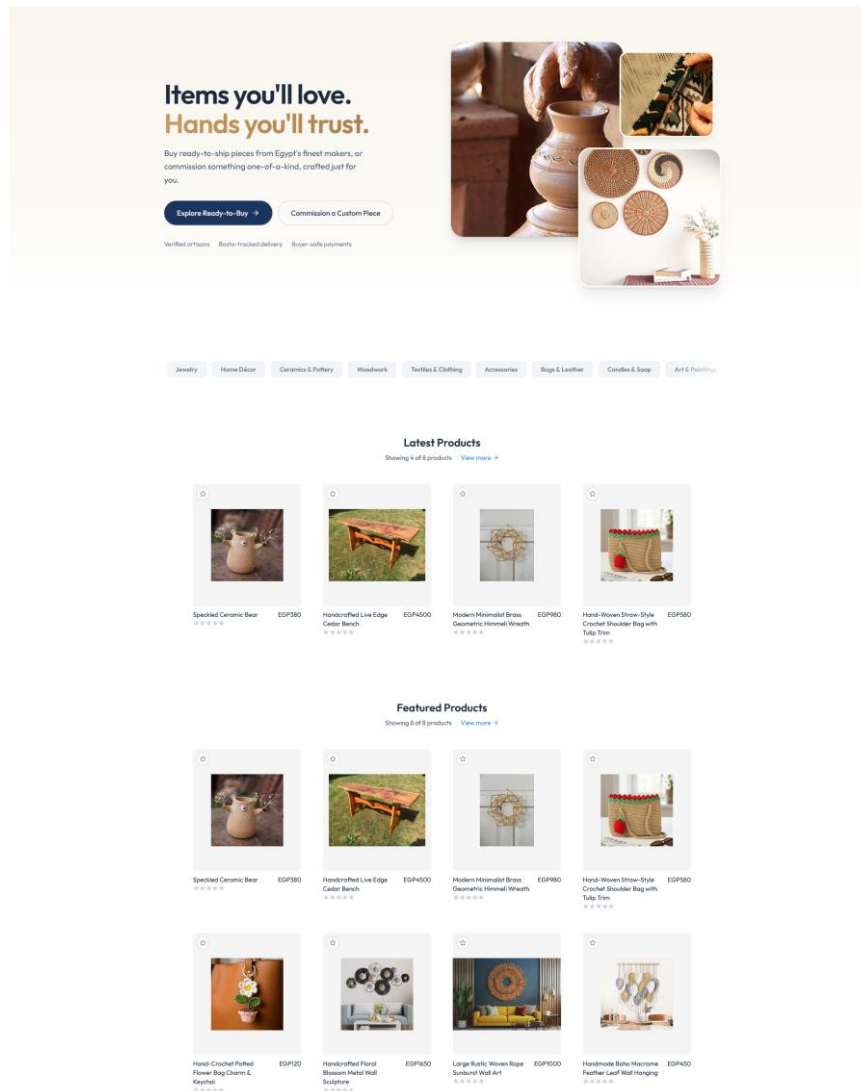
The screenshot shows a web form titled "Admin sign in" with the subtitle "Manzili administration portal". The form is centered on a light beige background. It contains two input fields: "Username" with a placeholder "e.g. yossef" and "Password". Below these fields is a dark blue "Sign in" button. At the bottom of the form, there is a link "Back to home" with a right-pointing arrow.

- Description: Separate, secured sign-in for administrators and moderators.
- Components: Email and password fields; "Login" button.
- Navigation: On success, opens the Admin dashboard (/admin). Non-admins are blocked.

5.7.3 Buyer Interface

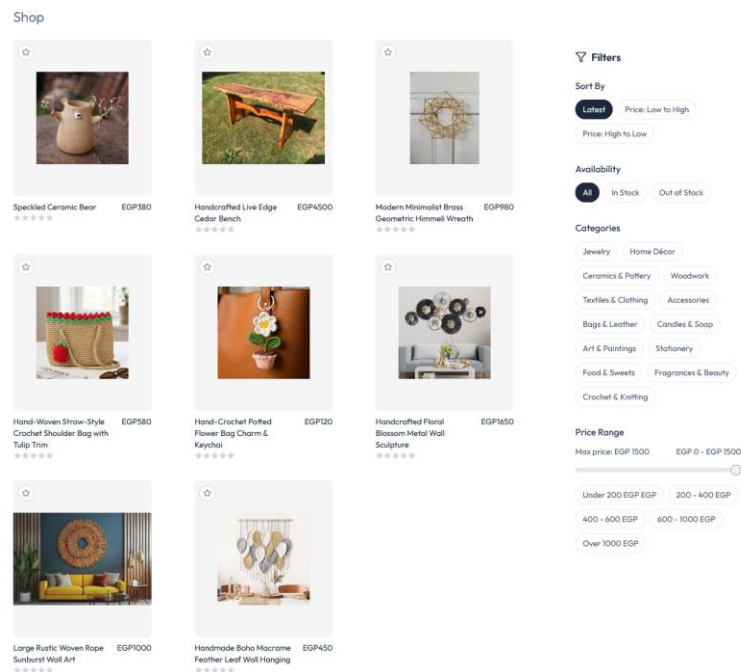
The buyer interface is the primary public-facing area of the Manzili platform, accessible to both guests and authenticated customers. It covers product discovery, purchasing, custom commissioning, and account management.

Home (/)



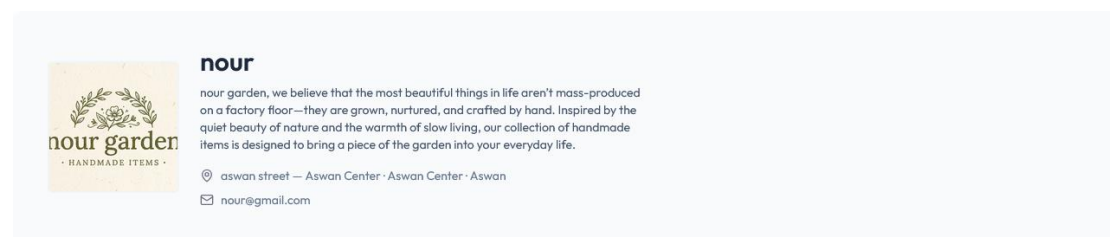
- **Description:** The landing page introducing Manzili and showcasing featured handmade products.
- **Components:** Hero section; category shortcuts; featured/latest product grids; call-to-action to shop or sell.
- **Navigation:** Links to the Shop, individual product pages, store pages, and the custom-requests area.

Shop (/shop)

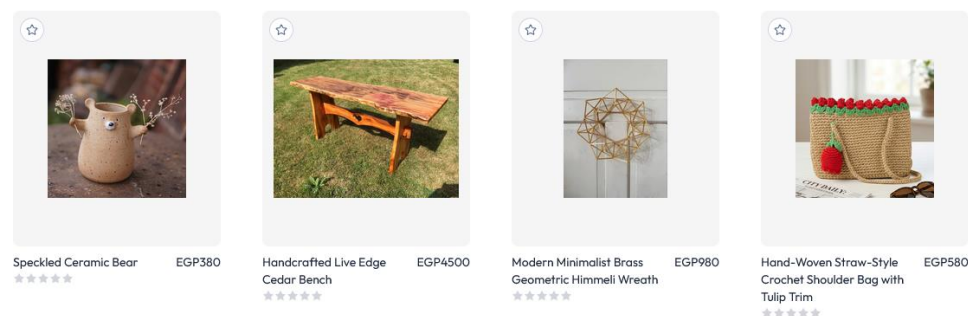


- Description: Browse and search all approved products across every store.
- Components: Search bar; category and sort filters; responsive product grid; pagination.
- Navigation: Each product card links to its product page (/product/[id]).

Storefront (/shop/[storeSlug])



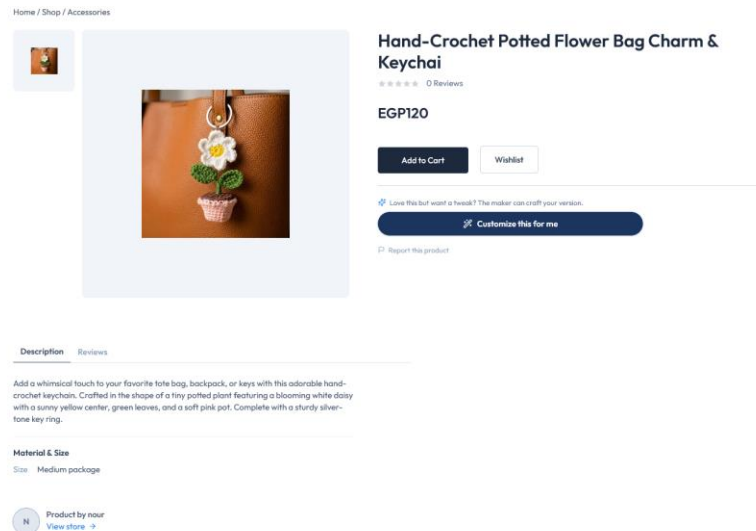
Shop



- Description: A single vendor's public store page.

- Components: Store logo, name, and description; the store's product grid; follow button; a "Custom Work" showcase of the store's completed bespoke commissions (each with the buyer's rating). The showcase is hidden when the store has no completed custom work.
- Navigation: Product cards link to their product pages.

Product Details (/shop/[storeSlug]/[productSlug])



- Description: Full details of one product.
- Components: Image gallery; price; description; variant selectors (e.g. size, colour) with per-option stock and pricing — the displayed price updates with the selected options; quantity selector; "Add to Cart" and wishlist buttons; ratings and reviews; link to the owning store; report button.
- Navigation: "Add to Cart" updates the cart; "View Store" opens the storefront; checkout flow starts from the cart.

Cart & Checkout (/cart and /checkout)

Shopping Cart

items in your cart [Add more](#) →

Product	Quantity	Total Price	Remove
 Hand-Crochet Potted Flower Bag Charm & Keychain Accessories EGP120	- 1 +	EGP120	

Payment Summary

Payment Method

☐ COD

☒ Card (Stripe)

☐ Mobile Wallet

Address

[Add Address +](#)

Subtotal: EGP120

Shipping (your 25%): EGP12.50

Processing fee: EGP3.84

You pay 25% of Bosta delivery + the card fee; the seller covers the other 75%.

Total (incl. est. shipping): EGP136.34

- Description: Review items before checkout.
- Components: Line items grouped by store with quantity controls; coupon code field; order summary (subtotal, shipping, discount, total); delivery-address selector; payment-method choice (cash on delivery, card, or mobile wallet).
- Navigation: "Checkout" places a cash-on-delivery order, or redirects to the chosen provider's secure page — Stripe for card payments or Kashier for mobile-wallet payments — then to the Success page.

Wishlist (/wishlist)

My Wishlist
2 items saved [Continue Shopping](#) →

 Handcrafted Live Edge Cedar Bench Home Décor EGP4500	Add to Cart Remove
 Handmade Boho Macrame Feather Leaf Wall Hanging Home Décor EGP450	Add to Cart Remove

- Description: Products the customer saved for later.
- Components: Saved-product grid; "move to cart" and "remove" actions. Hidden when logged out.
- Navigation: Product cards link to product pages; items can be moved to the cart.

My Orders (/orders)

My Orders

Showing total 1 orders [Home](#) →

Product	Total Price	Address	Status
EGP132.5	...		order placed Placed Processing Shipped Delivered

- Description: The customer's order history and live tracking.
- Components: List of orders; per-order status timeline (placed → processing → shipped → delivered); item details; tracking number; link to start a return.
- Navigation: Links to the Returns page; tracking timeline reflects Bosta updates.

Returns (/returns)

Returns & Refunds

Manage returns for your delivered orders [View all my orders](#) →

Return Policy

Items must be returned within 7 days of delivery.
Refunds are processed within 5-7 business days after the returned item is received.
Only card (Stripe) payments are eligible for online returns
COD orders must be resolved by contacting support
Items must be unused and in original packaging.



No delivered orders found
Only delivered orders are shown here.

[View all my orders](#) →

- Description: Request and track product returns.
- Components: Eligible delivered orders; return reason field; proof-image upload; status of existing return requests.
- Navigation: Submitting creates a return tied to the order; the request awaits admin approval before reverse logistics begin, and its status updates appear here.

Profile (/profile)

My Profile

Manage personal details, addresses, and custom orders in one place.

Personal data

Addresses

Custom requests

Personal data



profile.changePhoto

Name

nour

Email

nour@gmail.com

Save Changes

- Description: Manage the user's account and personal details.
- Components: Avatar upload; name/contact fields; saved addresses; account settings.
- Navigation: Saves changes to the account; addresses feed into checkout.

Custom Requests Hub (/custom-products)

Custom Products

No custom requests match your filters.
Be the first to create a custom request!

+ New Custom Request

+ Add New Request

Filters

Sort By

Latest Oldest First

Categories

Jewelry Home Décor
Ceramics & Pottery Woodwork
Textiles & Clothing Accessories
Bags & Leather Candles & Soap
Art & Paintings Stationery
Food & Sweets Fragrances & Beauty
Crochet & Knitting

Show

Open Requests My Requests

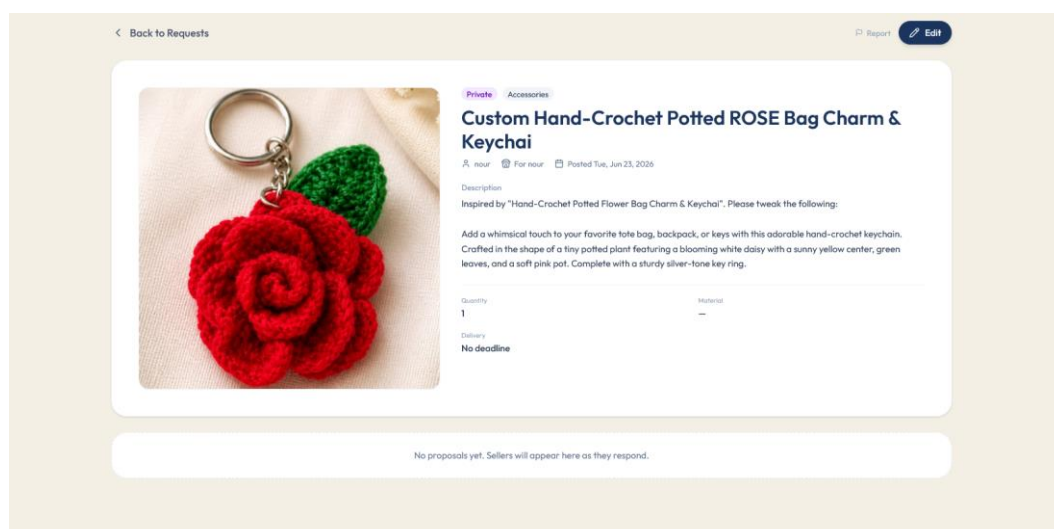
- Description: Entry point for Manzili's bespoke-commission feature; browse open requests and start a new one.

- Components: Grid of open custom requests; "New custom request" button; filters.
- Navigation: "New request" opens the custom form; each card opens a request view.

New Custom Request (/custom-products/new)

- Description: Submit a bespoke handmade request.
- Components: Item name, description, category, quantity, size, material, desired delivery date; reference image upload, colour palette picker, and voice-note recorder; visibility toggle (open to all vs. directed to one store). Requires login.
- Navigation: On submit, the request appears in the hub and in the relevant sellers' dashboards. Editing an existing request offers a delete option.

Request View (/custom-products/[requestId])



- **Description:** The buyer's workspace for a commission, covering its whole lifecycle.
- **Components:** The full request specification (details, images, colours, voice note); the seller's offer; the milestone payment steps (paid by card via Stripe or by mobile wallet via Kashier, with a delivery address captured at the final milestone). Once the final milestone is paid, a delivery-tracking timeline appears; after delivery, the buyer is prompted to leave a review.
- **Navigation:** Milestone payments redirect to the payment provider and return here; a completed, reviewed piece is added to the store's "Custom Work" showcase.

Offer Negotiation (/custom-products/[requestId]/offers/[offerId])

The screenshot displays the 'Offer Negotiation' interface. At the top, there's a navigation bar with 'Back to Requests' on the left and 'Report' on the right. The main content area is divided into two sections. The first section, 'REQUEST DETAILS', shows a red crocheted flower keychain. It includes a 'Private' label, a category 'Accessories', and a title 'Custom Hand-Crochet Potted ROSE Bag Charm & Keychai'. Below the image, there's a description: 'Inspired by "Hand-Crochet Potted Flower Bag Charm & Keychai". Please tweak the following: Add a whimsical touch to your favorite tote bag, backpack, or keys with this adorable hand-crochet keychain. Crafted in the shape of a tiny potted plant featuring a blooming white daisy with a sunny yellow center, green leaves, and a soft pink pot. Complete with a sturdy silver-tone key ring.' There are fields for 'Quantity' (set to 1), 'Delivery', and 'Colors' (set to 'Not specified'). The second section, 'Your Proposal', has a toggle for 'Accept buyer's date (not set)' which is turned on. It includes a 'Delivery' date picker (mm / dd / yyyy) and a 'Price (EGP)' field (0.00). There's a 'Note to buyer' text area with a character count '0/240'. At the bottom is a large orange 'Send Proposal' button. A small disclaimer at the very bottom reads: 'By negotiating, you commit to this price and deadline.'

- **Description:** Where a buyer reviews a seller's offer and negotiates.
- **Components:** Offer price and terms; per-offer chat; accept/decline actions.
- **Navigation:** Accepting an offer advances the commission to its milestone payments in the request view.

Collaboration Workspace (/custom-products/[requestId]/offers/[offerId]/workspace)

[Back to Requests](#)[Report](#)

**nour**
Accessories · **nour**

Accepted · **pay 1st**

Inspired by "Hand-Crochet Potted Flower Bag Charm & Keychain", Please tweak the following:
Add a whimsical touch to your favorite tote bag, backpack, or keys with this adorable hand-crochet keychain. Crafted in the shape of a tiny potted plant featuring a blooming white daisy with a sunny yellow center, green leaves, and a soft pink pot. Complete with a sturdy silver-tone key ring.

Type your message...

REQUEST DETAILS



Custom Hand-Crochet Potted ROSE Bag Charm & Keychain
Private · Accessories
1 hour · 1 hour · Jun 23, 2026

Description

Inspired by "Hand-Crochet Potted Flower Bag Charm & Keychain", Please tweak the following:

Add a whimsical touch to your favorite tote bag, backpack, or keys with this adorable hand-crochet keychain. Crafted in the shape of ...

Your Proposal

Accepted

Accept buyer's date (not set)

Delivery

Price (EGP)

Note to buyer (similar color, different material, etc.)

Optional — anything the buyer should know

0/240

Waiting for buyer's first payment

Awaiting buyer's first payment.

[Back to Requests](#)[Report](#)

**Pay the first half to start the work**
The artisan begins after the first half clears.

1/2 · Start
EGP 200.00

2/2 · Delivery
EGP 200.00

PAYMENT METHOD

☐ Cash on delivery (unavailable)

☒ Card (Stripe)

☐ Mobile Wallet

SUMMARY

Item	Custom Hand-Crochet Potted ROSE Bag Charm & Keychain
Seller	nour
Order subtotal	EGP 400.00
Already paid	EGP 0.00
First installment (1/2)	EGP 200.00

Due now

EGP 200.00

✓ Pay first installment

Sandbox demo — no real charge is made.
Next: EGP 200.00

**nour**
Accepted · EGP400 · due —

Accepted · **pay 1st**

Inspired by "Hand-Crochet Potted Flower Bag Charm & Keychain", Please tweak the following:
Add a whimsical touch to your favorite tote bag, backpack, or keys with this adorable hand-crochet keychain. Crafted in the shape of a tiny potted plant featuring a blooming white daisy with a sunny yellow center, green leaves, and a soft pink pot. Complete with a sturdy silver-tone key ring.

Type your message...

REQUEST DETAILS



Custom Hand-Crochet Potted ROSE Bag Charm & Keychain
Private · Accessories
1 hour · 1 hour · Tue, Jun 23, 2026

Description

Inspired by "Hand-Crochet Potted Flower Bag Charm & Keychain", Please tweak the following:

Add a whimsical touch to your favorite tote bag, backpack, or keys with this adorable hand-crochet keychain. Crafted in the shape of ...

Quantity

Material

Delivery

No deadline

- **Description:** The direct chat space between a buyer and seller around a request.
- **Components:** Request summary (item, colours, deadline, reference images, voice note); chat with image and voice attachments; the seller's offer is sent from here.
- **Navigation:** Sending or accepting an offer carries the commission into the request view, where payments and tracking happen.

Become a Vendor (/become-a-vendor)

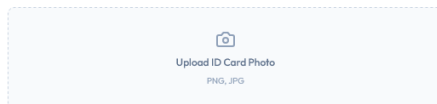
Create Your Store

Start selling your handmade products

Store Logo



National ID Card Photo (Required for Verification)



Store Name

your_store_name

Owner / Contact Person Name

yossef buyer

Description

Enter your store description

Email

yossefbuyer@gmail.com

Contact Number

Enter your store contact number

Address

Pick governorate, zone and district from Bosta so shipments validate – same flow as the checkout address.

City / governorate

Select city

Zone

Select zone

District

Select district

Street / address line

Building

Floor

Apartment

Postal code (optional)

Submit

- **Description:** Application form to open a store on Manzili.

- **Components:** Store name, username, description, logo upload; contact and address fields; national-ID image upload (required for verification); submit button.
- **Navigation:** On submit, the application is sent to admins for review; the store stays inactive until approved.

Informational Pages

Manzili Policies

Policies & Privacy

Manzili is a marketplace for handmade Egyptian products, connecting makers and buyers. Here is exactly how our fees, custom orders, data handling, payment holds, and returns work — in plain language.

Last updated: June 2026

ON THIS PAGE

1. Fees & Commissions
2. Custom Orders & Creative Rights
3. What Administrators Can See
4. Security & Your Data
5. Payment Holds & Returns Reserve
6. Returns & Refunds
7. What You Can Sell

Questions?
Email us at manziliproject@gmail.com

1 Fees & Commissions

Browsing and listing on Manzili are free. We only earn when a sale goes through. Here is every charge that can apply to an order and who pays it.

Charge	Who pays	Amount
Manzili commission — standard orders	Seller	15% of item price
Manzili commission — custom "customize this for me" orders	Seller	10% of item price
Bosta delivery fee	Split: seller & buyer	75% seller / 25% buyer
Stripe card processing fee	Buyer	~2.9%

WORKED EXAMPLE — A 500 EGP ITEM

Buyer pays		Seller nets	
Item price	500.00	Item price	500.00
Shipping (25% of 50 EGP)	12.50	- Manzili commission (15%)	-75.00
Stripe fee (~2.9%)	14.86	- Shipping (75% of 50 EGP)	-37.50
Total	527.36 EGP	Net earnings	387.50 EGP

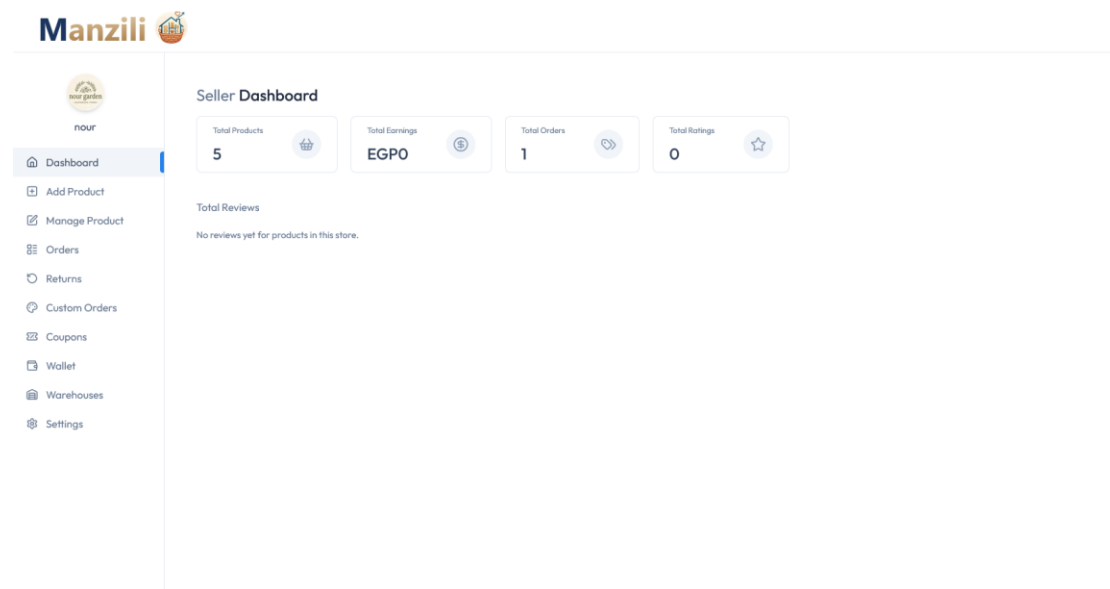
Manzili includes a set of static informational pages accessible from the footer navigation:

- **Description:** Informational pages — the Manzili story, a contact form, common questions, and the policy/terms (fees, creative rights, security, returns).
- **Components:** Static content; the Contact page includes a form that sends an email.
- **Navigation:** Linked from the footer; the Contact form submits to the backend email service `./privacy-policy` — presents the platform's privacy policy covering data collection, storage, and usage.

5.7.4 Vendor Dashboard

The vendor dashboard is the dedicated management interface for artisans and makers operating storefronts on the Manzili platform. It is accessible exclusively to authenticated users who have completed the vendor registration process and received admin approval. The dashboard is organized as a sidebar-navigated panel, with each section covering a distinct area of store management.

Dashboard (/store)



- Description: The vendor's home screen with an overview of store activity.
- Components: Summary cards (orders, products, balance); recent activity; sidebar navigation.
- Navigation: Links to every section of the dashboard..

Add Product (/store/products/new)

Manzili

nour

Dashboard

Add Product

Manage Product

Orders

Returns

Custom Orders

Coupons

Wallet

Warehouses

Settings

Add New Product

Product images

Name

Product name

Description

Description

Material

e.g. Oak wood, cotton, ceramic

Shown on the product page under the description.

Price (EGP)

0

Sale price (EGP)

—

Stock (units)

0

Optional — only if the item is discounted (below the price).

How many units you have to sell.

Package size (internal)

Medium

Bulky / type profile

Normal

- Description: Create a new product listing.
- Components: Form for name, description, price, stock, category, shipping size, image upload, and variant options (e.g. size, colour) with per-option stock and price adjustment; "Save" button.
- Navigation: On save, the product appears in Manage Products and the storefront..

Manage Products (/store/products)

Manzili

nour

Dashboard

Add Product

Manage Product

Orders

Returns

Custom Orders

Coupons

Wallet

Warehouses

Settings

Manage Products

Search

Search by name...

Category

All categories

Status

All

Sort

Newest

PRODUCT	CATEGORY	PRICE	STOCK	SOLD	STATUS	ACTIONS
 Speckled Ceramic Bear	Home Décor	EGP 380	2 Low	0	Low stock	Edit  
 Handcrafted Live Edge Cedar Bench	Home Décor	EGP 4,500	1 Low	0	Low stock	Edit  
 Modern Minimalist Brass Geometric Himmel Wreath	Home Décor	EGP 980	4 Low	0	Low stock	Edit  
 Hand-Woven Straw-Style Crochet Shoulder Bag with Tulip Trim	Bags & Leather	EGP 580	4 Low	0	Low stock	Edit  
 Hand-Crochet Potted Flower Bag Charm & Keychain	Accessories	EGP 120	14	1	In stock	Edit  

- Description: View and maintain the store's products.

- Components: Product table/grid; edit and delete actions; enable/disable visibility toggle.
- Navigation: Edit links to the product editor; deletes remove the listing (with confirmation).

Store Orders (/store/orders)

The screenshot displays the 'Store Orders' page for a user named 'nour'. The interface includes a sidebar with navigation options: Dashboard, Add Product, Manage Product, Orders (selected), Returns, Custom Orders, Coupons, Wallet, Warehouses, and Settings. The main content area shows details for order #10, dated 6/23/2026, 4:08:24 AM, with a status of 'Processing'.

Order Details:

- Customer:** Alexandria
- Items:** 0 pieces
- Payment:** STRIPE, Paid (checked), Total: \$380.00
- Shipping & label:** Carrier: Bosta, Shipment status: CREATED, Tracking #: HZ000010233. A QR code is provided for tracking.
- Tracking history:** Bosta HZ000010233. Status: Order confirmed — Bosta pickup requested from the seller. 23 Jun, 04:08.

Below the main order details, there is a section for the 'Order waybill' which includes a QR code, tracking number HZ000010233, and a table of items:

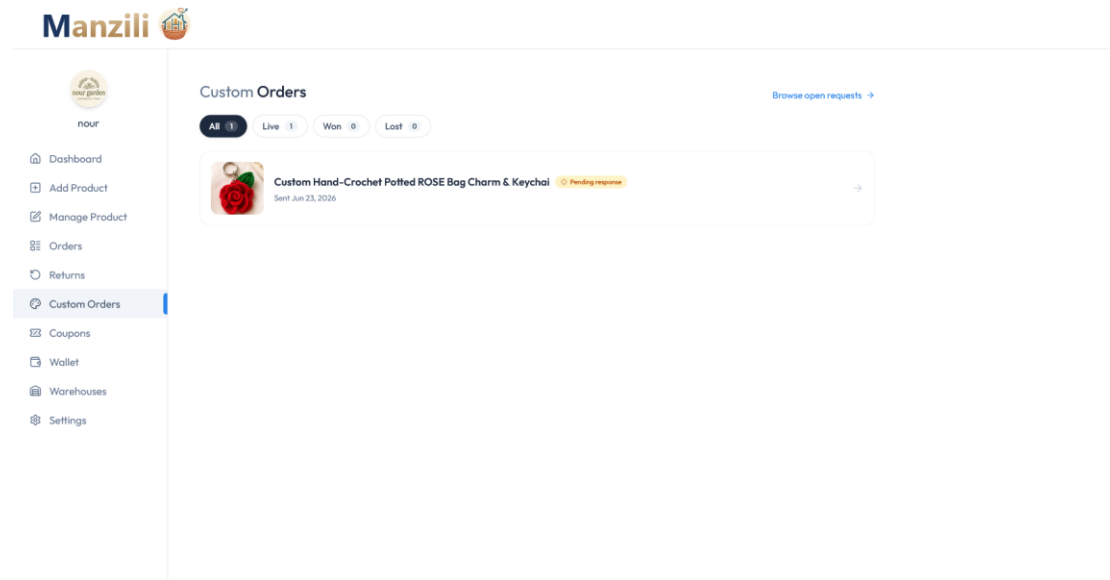
ITEM	QTY	PRICE
Total		\$380.00

The bottom of the page shows a navigation bar with a 'Print' button and a 'Save' button. The page is titled 'Manzili - Store Dashboard' and the URL is 'http://localhost:3000/store/orders'.

- Description: Incoming orders for this store to fulfil.

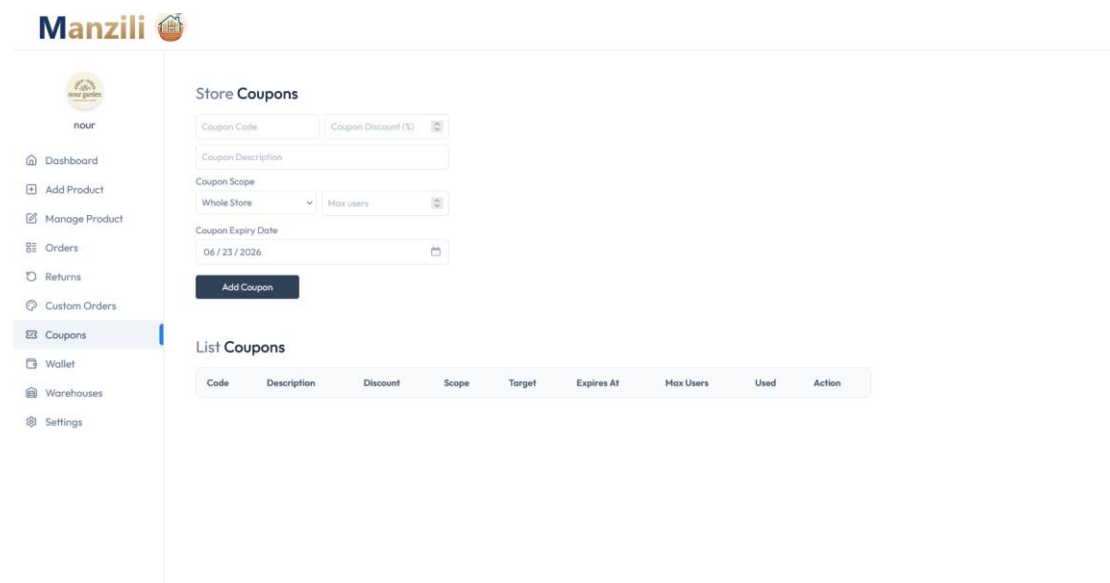
- Components: Order list with customer and item details; a single "Confirm & request pickup" action that creates the Bosta shipment; status timeline.
- Navigation: Confirming an order triggers courier pickup; tracking appears on the order.

Custom Orders (/store/custom-orders)



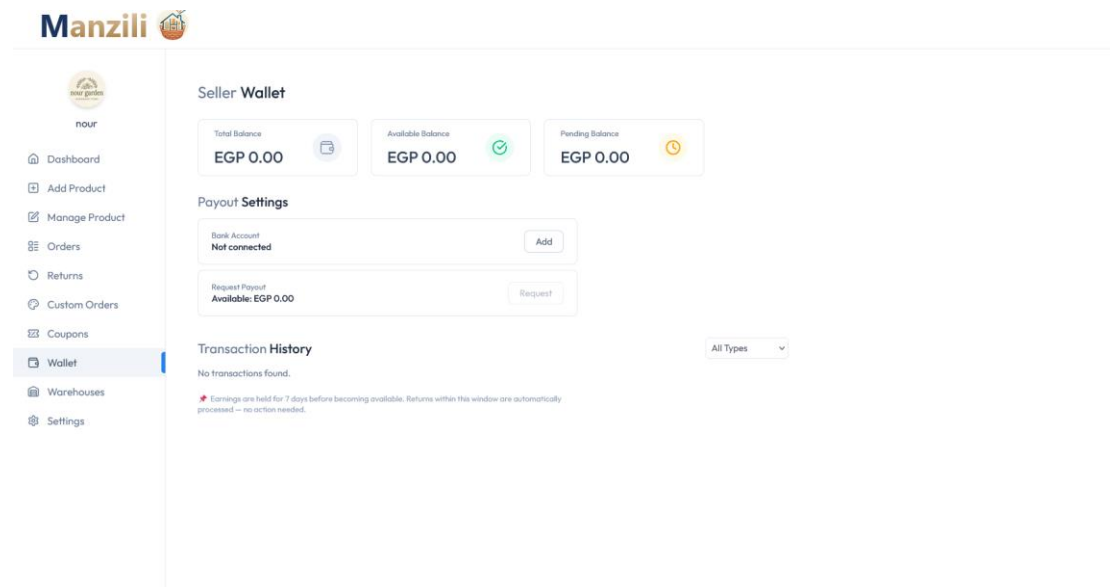
- Description: Custom requests directed to this store, plus open requests it has engaged with.
- Components: List of relevant requests; "send offer" action; offer chat.
- Navigation: Sending an offer opens the negotiation thread with the buyer.

Coupons (/store/coupons)



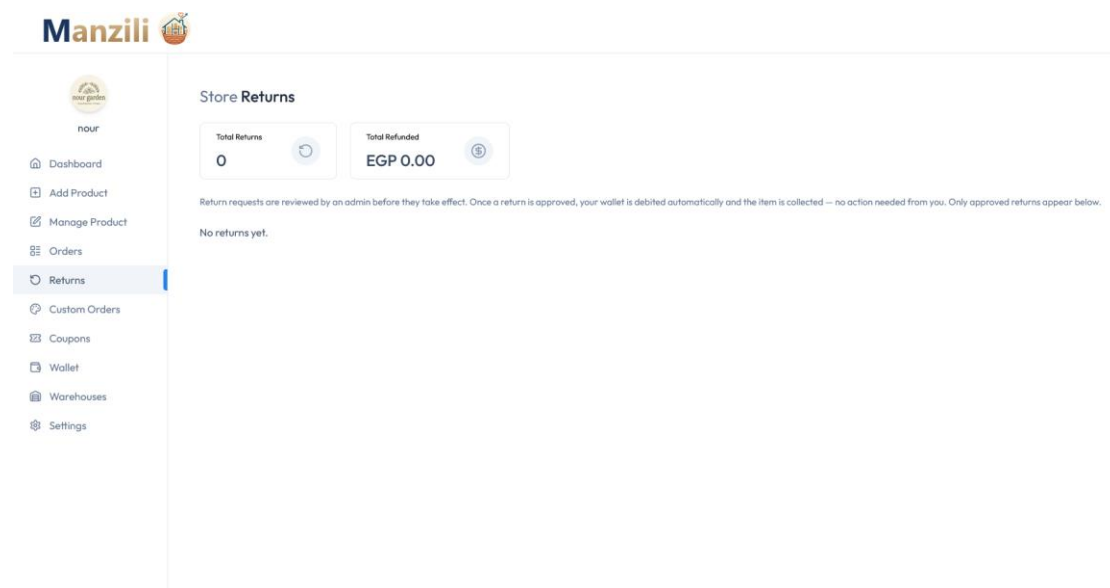
- Description: Create and manage discount codes for the store.
- Components: Coupon list; create form (code, discount, limits, expiry); enable/disable.
- Navigation: Saved coupons become usable by buyers at checkout.

Wallet (/store/wallet)



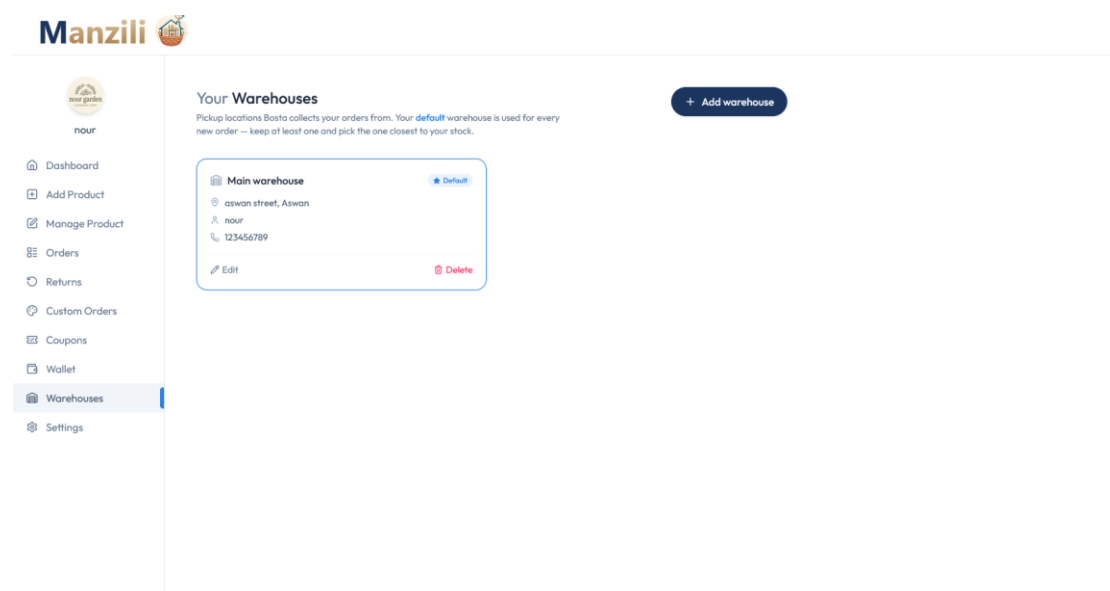
- Description: The seller's earnings and payout area.
- Components: Pending vs available balance; transaction history; "Request payout" with bank details.
- Navigation: Payout requests are recorded for admin processing.

Returns (/store/returns)



- Description: Return requests raised against this store's orders.
- Components: Return list with reasons and proof images; approve/process actions.
- Navigation: Processing a return updates its status and triggers reverse logistics.

Warehouses (/store/warehouses)



- Description: Manage the store's pickup locations for courier collection.
- Components: Warehouse list; add/edit form (address, city, phone, contact, label); default-location toggle.

- Navigation: The default warehouse is used as the pickup point when shipments are created.

Settings (/store/settings)

Manzili

Store Settings
Update how your shop appears across Manzili.

Store Logo

Username

Name

Description

Email

Contact Number

Public location (shown on your storefront)

- Description: Edit store identity and payout details.
- Components: Store name, description, logo, contact, and financial/bank fields; save button.
- Navigation: Changes apply to the public storefront and payout records.

5.7.5 Admin Panel

The admin panel is the platform governance interface for Manzili system administrators and moderators. It is accessible exclusively through the dedicated admin login screen at /admin/login and is entirely separate from the buyer and vendor interfaces. The panel is organized as a sidebar-navigated dashboard, with access to specific sections controlled by the moderator permission system — full administrators have unrestricted access to all sections, while moderators are granted access only to the sections explicitly assigned to them by a full administrator.

Admin Dashboard (/admin)

The screenshot displays the Manzili Admin Dashboard. At the top, the Manzili logo is visible. Below it, a sidebar on the left contains navigation links: Dashboard, Stores, Approve, Products, Orders, Returns, Custom Requests, Reports, Coupons, Conversations, and Moderators. The main content area is titled 'All Stores' and features a table with columns: Sr., Store, Owner, Status, Active, Products, and Actions. The table lists three stores: 1. 'yossef' (status: pending), 2. 'nour' (status: approved), and 3. 'بوت اوي' (status: approved). Each store entry includes a 'View' button and an 'Approve' or 'Delete' button. The 'Active' column shows toggle switches for each store.

Sr.	Store	Owner	Status	Active	Products	Actions
1	yossef @py_fashion	yossef yossef	pending	☐	0	View Approve Reject
2	nour @nour_handmade_garden	nour nour@gmail.com	approved	☒	0	View Delete
3	بوت اوي @shop	بوت اوي deco@gmail.com	approved	☒	0	View Delete

- Description: Overview of platform-wide statistics and activity.
- Components: Summary cards (users, stores, orders, and — for full admins only — Manzili's commission revenue); recent activity; sidebar.
- Navigation: Links to all admin sections.

Chapter 6: Testing & Evaluation

6.1 Testing Strategy

Manzili followed a phased, iteration-driven testing approach rather than a rigid, predefined test plan. Testing progressed through four distinct stages: prototyping with cookie-based local storage, frontend deployment on Vercel while still relying on that same storage layer, migration to a real backend connected to Supabase, and finally containerization and deployment on AWS. Throughout each stage, the team followed a consistent rhythm of code, deploy, review, repeat — changes were committed to GitHub, exercised by the team, and refined based on observed behavior before moving forward.

Several supporting tools were used throughout this process: Vercel for rapid frontend preview deployments, Apidog for API design and endpoint testing, and ana.hsoub.com for bug reporting and feature tracking. This phased approach allowed the team to validate the platform incrementally at each layer of the stack, rather than relying on a single, late-stage testing phase.

6.2 Cookie-Based Prototyping on Localhost

The earliest stage of testing was conducted entirely on localhost using mock data and, where necessary, a minimal local backend. Application state during this phase was persisted in the browser through cookies and local storage rather than a remote database.

This setup allowed the team to validate core screens, navigation flows, and complete user workflows — including the custom order flow — across all platform roles (customer, vendor, and admin) early in development, without depending on any production infrastructure. This stage served as a fast feedback loop for catching usability and logic issues before backend development began.

6.3 Integration Testing

Following local prototyping, the frontend repository was connected to Vercel for automatic preview deployments. At this stage, the deployed frontend still relied on the same cookie-based local storage layer used in localhost testing — it was not yet connected to a live backend.

6.4 Backend Migration, API Testing, and Supabase

Once frontend flows were validated on Vercel, the application was migrated off cookie-based local storage and connected to a real backend. This required every previously validated feature to be retested against live API responses rather than mock client-side data.

API endpoints were designed, debugged, and tested using Apidog, as documented in Section 4.4. When a frontend action failed during this phase, developers reproduced the corresponding request directly in Apidog to isolate whether the fault originated in the endpoint logic, authentication handling, or client-side request handling.

Supabase was validated during this phase as the production database layer, confirming that data persistence, relationships, and queries behaved correctly before the team proceeded to the final deployment stage.

6.5 Results

The phased testing process — beginning with cookie-based prototyping, followed by Vercel frontend testing on that same storage layer, backend integration through a separate repository, Apidog-based API validation, Supabase verification, and finally containerization and hosting on AWS — allowed the team to validate Manzili incrementally at every layer of the stack, without relying on a single late-stage testing phase.

This approach proved well-suited to a large team working under time constraints, as it allowed parallel progress on frontend and backend development while maintaining a continuous feedback loop. However, it relied entirely on manual verification rather than automated regression test suites, which is acknowledged as a limitation and addressed as a proposed future enhancement in Chapter 7.

Chapter 7: Conclusion & Future Work

7.1 Summary of Achievements

This project set out to address a persistent gap in the Egyptian retail landscape: the absence of a dedicated, purpose-built marketplace for handmade and artisanal products. General e-commerce platforms force craftsmen to compete on price with mass-produced goods, while social media offers visibility without the infrastructure needed for secure transactions, order management, or sustainable business growth. Manzili was conceived and delivered as a direct response to this problem — a specialized multi-vendor e-commerce platform that connects skilled Egyptian artisans with customers who actively seek and value handcrafted goods.

Over the course of this graduation project, the team successfully designed, implemented, and deployed a fully functional platform that brings together the core capabilities required by such a marketplace. Manzili supports three distinct user roles — customers, vendors, and administrators — each with tailored interfaces and workflows. Vendors operate independent storefronts through a dual-model approach: a gallery of ready-to-buy products for immediate purchase, and a custom request portal where customers can collaborate directly with makers on bespoke or personalized items. This dual model reflects the real-world nature of handmade commerce, where both off-the-shelf and made-to-order work coexist.

On the technical side, the platform was built using a modern full-stack architecture. The frontend was developed with Next.js and Tailwind CSS, delivering a responsive and intuitive experience across desktop and mobile browsers. The backend exposes a RESTful API built on ASP.NET Core, handling authentication, product management, order processing, custom request workflows, in-platform messaging, and integration with third-party services. Persistent data is stored in a PostgreSQL database managed through Supabase, which also provides authentication support, real-time capabilities for features such as live chat and order status updates, and file storage for product and custom-request media.

Beyond the core e-commerce functionality, Manzili integrates several external services that extend the platform's capabilities. Stripe was integrated for secure online payment processing. Bosta was adopted for delivery logistics and shipment tracking within Egypt. The Gemini API was incorporated to power AI-assisted features within the platform. Mobile accessibility was achieved through a native Android application that wraps the web application inside a WebView shell, allowing users to access the platform on the go without maintaining a separate mobile codebase. The entire application stack was containerized using Docker, with the frontend deployed on Vercel and the backend hosted on AWS cloud infrastructure, ensuring consistent behavior across environments and a foundation for future scalability.

Additional features that distinguish Manzili from general marketplaces include direct buyer-seller chat for building trust and coordinating custom orders, order tracking for both customers and vendors, and a vendor dashboard for store and product management. The development process followed a structured software engineering approach — from system analysis and requirements gathering, through database and API design, to frontend and backend implementation and testing — resulting in a cohesive platform that uniquely positions itself within the Egyptian market.

In summary, Manzili successfully transforms the concept of a handmade-focused marketplace from an identified need into a working system. It provides artisans with a centralized platform to showcase their craft without competing against factory-made alternatives, gives customers a dedicated space to discover and purchase unique handcrafted goods, and demonstrates that a graduation project team can deliver a scalable, secure, and user-friendly e-commerce ecosystem from the ground up.

7.2 Limitations

Despite the achievements outlined above, this project was not without its constraints. Acknowledging these limitations is essential for an honest evaluation of the work and for guiding future development.

AI Integration Constraints: The platform incorporates AI-powered features through the Gemini API, as well as supplementary AI services including z.ai and puter.js. However, the team was unable to obtain access to more advanced domain-specific or frontier AI models that might have produced higher-quality, more contextually relevant content. As a result, the AI-generated outputs occasionally fell short of the quality standards the team had envisioned. The available models, while functional, did not always achieve the level of quality needed for a fully polished user experience.

Mobile Application Strategy: The original project plan included building a fully native mobile application using Flutter. Two team members had experience as Flutter developers, and the initial intention was to deliver a dedicated cross-platform mobile app alongside the web application. However, as development progressed and the team began scoping the mobile implementation in earnest, the full complexity of that path became apparent. Managing Flutter version compatibility, navigating the framework's tooling overhead, and — most critically — sustaining a consistent user experience, styling, and feature parity between the Flutter application and the existing Next.js web application would have required significant additional effort and ongoing maintenance. Rather than abandoning mobile accessibility entirely, the team pivoted to a WebView-based approach: a native Android shell that wraps the web application, providing mobile access while avoiding the burden of a separate codebase. This decision was pragmatic given the project's timeline and resources, but it means the mobile experience is fundamentally tied to the web application's rendering and does not leverage native mobile capabilities such as offline access, push notifications, or platform-specific UI patterns to their full potential.

System Diagrams: The documentation of system analysis and design includes context-level and Level 1 data flow diagrams, a use case diagram, an activity diagram, an entity relationship diagram, and architecture overviews. However, the team was unable to produce child-level data flow diagrams (Diagram Level 2) or use case sequence diagrams that met the quality bar required for inclusion in the final report. Several attempts were made to develop these diagrams, but time constraints and the team's limited experience in formal diagramming techniques meant that none of the drafts reached a level of clarity and completeness suitable for publication. While the existing diagrams adequately convey the system's structure and major interactions, the absence of more granular diagrammatic documentation represents a gap in the formal analysis deliverables.

7.3 Future Enhancements

The current version of Manzili provides a solid foundation for a specialized handmade marketplace. However, several areas have been identified for future development that would significantly strengthen the platform's relevance and usability within the Egyptian market.

Local Payment Integration: The platform currently relies on Stripe for payment processing, which serves international card transactions well but does not cover the full spectrum of payment methods commonly used by Egyptian consumers. A key future enhancement is the integration of local payment APIs, including InstaPay, Vodafone Cash, and Fawry. These services are widely adopted across Egypt and would lower the barrier to entry for customers who prefer mobile wallets, bank transfers, or cash-based payment options over international credit cards.

Specialized Delivery for Food and Fragile Items: Manzili currently uses Bosta as its delivery logistics partner, which provides reliable shipment tracking and nationwide coverage for standard product categories. However, Bosta's service guidelines explicitly prohibit certain item types, including perishable food and beverage products, as well as sensitive, fragile, or easily breakable items that require specialized packaging and handling. Given that the handmade industry encompasses categories such as home-made food, delicate ceramics, glasswork, and other fragile crafts, this limitation restricts the range of products vendors can safely list and ship through the platform. A future enhancement would involve either integrating with a local delivery service that supports these item categories, or establishing and sustaining the platform's own specialized fulfillment network for food and fragile goods. Such a capability would allow Manzili to serve a wider variety of artisan product types and remove a significant barrier for vendors whose goods fall outside standard courier guidelines.

7.4 Closing Remarks

Working on Manzili was, above all, a lesson in turning a real, observed problem into a working piece of software. What began as a frustration shared by Egyptian artisans — talented makers losing their craft's value to algorithms

built for mass-produced goods — gradually took shape into a full platform spanning database design, API architecture, frontend interfaces, and the countless small decisions that sit between an idea and a deployed system. Along the way, the team learned as much from the constraints and trade-offs, like the pivot away from Flutter, the limits of the available AI models, as from the features that worked exactly as planned. That kind of learning rarely shows up cleanly in a diagram, but it shaped this project just as much as any line of code did. Manzili in its current form is not the final word on what a dedicated handmade marketplace for Egypt could be, but it is a real, functioning step toward it, built by a team that chose to understand the problem first and the technology second. If even a handful of artisans find it easier to reach the customers who genuinely value their work because of what was built here, the project will have done what it set out to do.

Appendices

Appendix A:

Codebase:

<https://github.com/yossefAraby/Manzili>



Mobile application download link via github:

<https://github.com/yossefAraby/Manzili/releases/download/MobileApp/Manzili.apk>



Live WebAppliaction:

<https://manzili-mis.vercel.app/>



Documentation digital version

